NEXT.JS

CHEAT SHEETS

A Complete Reference for Next.js 16 and the App Router

App Router · Server Components · Streaming · Cache Components · Server Actions · Route Handlers · proxy.ts · Metadata & SEO · Auth · Testing · Deployment

60 REFERENCE SHEETS

CONTENTS

INTRODUCTION & SETUP

01 Introduction & Rendering Model
02 Creating a Project
03 Project Structure & Special Files
04 next.config.ts
05 CLI & Dev Tooling

ROUTING

06 File-System Routing
07 Dynamic Routes
08 Layouts & Templates
09 Linking & Navigation
10 Loading, Error & Not Found UI
11 Route Groups & Private Folders
12 Parallel & Intercepting Routes

SERVER & CLIENT COMPONENTS

13 Server Components
14 Client Components
15 Composition Patterns
16 React 19 Features Used in Next

DATA FETCHING

17 Fetching in Server Components
18 Streaming & Suspense
19 Client-Side Fetching
20 params & searchParams

CACHING & RENDERING

21 Static vs Dynamic Rendering
22 Cache Components & 'use cache'
23 cacheLife & cacheTag
24 Revalidation
25 Partial Prerendering
26 Previous Caching Model

MUTATIONS & SERVER ACTIONS

27 Server Actions Basics
28 Forms & Pending States
29 Server Action Security

ROUTE HANDLERS & PROXY

30 Route Handlers
31 Route Handler Patterns
32 proxy.ts (formerly middleware.ts)
33 Request-Time APIs

STYLING

34 CSS, CSS Modules & Sass
35 Tailwind CSS
36 CSS-in-JS & Component Libraries

OPTIMIZATIONS

37 next/image
38 next/font
39 Metadata API
40 File-Based Metadata & SEO
41 Scripts & Lazy Loading

AUTHENTICATION & SECURITY

42 Authentication Patterns
43 Protecting Routes & Data
44 Security Headers & CSP

DATA, ENVIRONMENT & I18N

45 Environment Variables
46 Databases & ORMs
47 Internationalization

TESTING & DEBUGGING

48 Unit & Component Testing
49 End-to-End Testing
50 Debugging

DEPLOYMENT & OPERATIONS

51 Builds & Build Output
52 Self-Hosting & Docker
53 Static Export
54 Platforms & Deployment Adapters
55 Instrumentation & Observability

PAGES ROUTER (LEGACY)

56 Pages Router Basics
57 Pages Router Data Fetching
58 API Routes & Migrating to App Router

QUICK REFERENCE

59 Quick Reference — Routing & Rendering
60 Quick Reference — Caching, Actions & Deployment

NEXT

Table of Contents

60 sheets · syntax · examples · notes

INTRODUCTION & SETUP 01 – 05

- 01 Introduction & Renderi…
- 02 Creating a Project
- 03 Project Structure & Sp…
- 04 next.config.ts
- 05 CLI & Dev Tooling

ROUTING 06 – 12

- 06 File-System Routing
- 07 Dynamic Routes
- 08 Layouts & Templates
- 09 Linking & Navigation
- 10 Loading, Error & Not Fo…
- 11 Route Groups & Privat…
- 12 Parallel & Intercepting …

SERVER & CLIENT COMPONENTS 13 – 16

- 13 Server Components
- 14 Client Components
- 15 Composition Patterns
- 16 React 19 Features Use…

DATA FETCHING 17 – 20

- 17 Fetching in Server Co…
- 18 Streaming & Suspense
- 19 Client-Side Fetching
- 20 params & searchPara…

CACHING & RENDERING 21 – 26

- 21 Static vs Dynamic Ren…
- 22 Cache Components & …
- 23 cacheLife & cacheTag
- 24 Revalidation
- 25 Partial Prerendering
- 26 Previous Caching Model

MUTATIONS & SERVER ACTIONS 27 – 29

- 27 Server Actions Basics
- 28 Forms & Pending States
- 29 Server Action Security

ROUTE HANDLERS & PROXY 30 – 33

- 30 Route Handlers
- 31 Route Handler Patterns
- 32 proxy.ts (formerly mid…
- 33 Request-Time APIs

STYLING 34 – 36

- 34 CSS, CSS Modules & S…
- 35 Tailwind CSS
- 36 CSS-in-JS & Compon…

OPTIMIZATIONS 37 – 41

- 37 next/image
- 38 next/font
- 39 Metadata API
- 40 File-Based Metadata …
- 41 Scripts & Lazy Loading

AUTHENTICATION & SECURITY 42 – 44

- 42 Authentication Patterns
- 43 Protecting Routes & D…
- 44 Security Headers & CSP

DATA, ENVIRONMENT & I18N 45 – 47

- 45 Environment Variables
- 46 Databases & ORMs
- 47 Internationalization

TESTING & DEBUGGING 48 – 50

- 48 Unit & Component Tes…
- 49 End-to-End Testing
- 50 Debugging

DEPLOYMENT & OPERATIONS 51 – 55

- 51 Builds & Build Output
- 52 Self-Hosting & Docker
- 53 Static Export
- 54 Platforms & Deployme…
- 55 Instrumentation & Obs…

PAGES ROUTER (LEGACY) 56 – 58

- 56 Pages Router Basics
- 57 Pages Router Data Fet…
- 58 API Routes & Migrating…

QUICK REFERENCE 59 – 60

- 59 Quick Reference — Ro…
- 60 Quick Reference — Ca…

Sheet 01

Introduction & Rendering Model

App Router · Pages Router · Server Components · Turbopack · self-hosting

WHAT NEXT.JS ADDS TO REACT

Next.js is a full-stack React framework. React gives you components; Next.js adds file-based **routing**, server rendering, data fetching, caching, bundling (**Turbopack**), image/font optimization, and a production server. The **App Router** (`app/`) is the current model: components render on the **server by default** and only opt into the browser with `'use client'`.

VERSION NOTES (16.X)

- 16.3 (Aug 2026); App Router bundles React 19.2+ canary
- Requires **Node.js 20.9+** and TypeScript **5.1+**
- Turbopack is the default bundler for `dev` and `build`
- Majors ship roughly yearly; minors every few months

HELLO, WORLD (APP ROUTER)

```tsx
// app/layout.tsx — required root layout
export default function RootLayout({ children }: LayoutProps<'/'>) {
  return (
    <html lang="en">
      <body>{children}</body>
    </html>
  )
}

// app/page.tsx — the "/" route (a Server Component)
export default async function Home() {
  const res = await fetch('https://api.github.com/repos/vercel/next.js')
  const repo = await res.json()
  return <h1>Hello, World — {repo.stargazers_count} stars</h1>
}
```

HOW IT RENDERS

- → `app/layout.tsx` — wraps every page; must render `<html>` and `<body>`
- → `app/page.tsx` — a file named `page` makes the folder a public URL
- → `async` component — fetches on the server; no `useEffect` needed
- → `LayoutProps<'/'>` — global type helper generated by Next.js
- → Output is HTML + an RSC payload; only Client Components ship JS

APP ROUTER VS PAGES ROUTER

FEATURE	APP ROUTER	PAGES ROUTER
Folder	`app/`	`pages/`
Default component	Server Component	Client (hydrated)
Data fetching	`async` components	`getServerSideProps`
Layouts	Nested `layout.tsx`	`_app.tsx` only
Mutations	Server Actions	API routes
Status	Recommended	Maintained (legacy)

WHERE IT RUNS

TARGET	NOTES
Vercel	Zero-config; verified adapter
`next start`	Any Node.js host; all features
Docker	`output: 'standalone'`
Static export	`output: 'export'`; no server features
Adapters	Bun (verified), Netlify, Cloudflare

⚠ COMMON MISTAKES

- Treating Next.js like a client-only SPA: fetching in `useEffect` throws away server rendering and adds a waterfall
- Adding `'use client'` to every file out of habit; it ships all that code to the browser
- Following Pages Router tutorials (`getServerSideProps`) inside `app/`; those APIs do not exist there

✓ TIPS & BEST PRACTICES

- Start new projects with the App Router; mix in `pages/` only during migrations
- Run `npx next info` when filing bugs; it prints Node, Next, and React versions
- Read version-matched docs from `node_modules/next/dist/docs` (linked from `AGENTS.md`)

QUICK SUMMARY

- `app/` — App Router, server-first rendering
- `page.tsx` — makes a route public
- `layout.tsx` — shared UI; root renders `<html>`
- `'use client'` — opt a subtree into the browser
- Turbopack — default bundler in 16.x

Creating a Project

SCAFFOLDING A NEW APP

`create-next-app` generates a ready-to-run project. Accepting the **recommended defaults** gives you TypeScript, ESLint, Tailwind CSS, the App Router, Turbopack, and an `AGENTS.md` file for coding agents. Every prompt also has a flag, so the setup can be scripted for CI or templates.

REQUIREMENTS

- **Node.js 20.9+** (LTS); Node 18 is no longer supported
- Works with npm, pnpm, Yarn, and Bun
- `--yes` reuses previous answers or the defaults
- `next dev` serves on `http://localhost:3000`

CREATE AND RUN

```
npx create-next-app@latest my-app            # interactive prompts
npx create-next-app@latest my-app --yes      # accept defaults
npx create-next-app@latest my-app --ts --tailwind --eslint \
  --app --src-dir --import-alias "@/*" --use-pnpm
npx create-next-app@latest api-only --api    # route handlers only
npx create-next-app@latest -e with-docker    # official example

cd my-app
npm run dev        # next dev   → http://localhost:3000
npm run build      # next build → .next/
npm run start      # next start (production server)
```

WHAT YOU GET

- → `package.json` scripts: `dev`, `build`, `start`, `lint`
- → `app/layout.tsx`, `app/page.tsx`, `app/globals.css`
- → `next.config.ts`, `tsconfig.json`, `eslint.config.mjs`
- → `AGENTS.md` + `CLAUDE.md` pointing agents at bundled docs
- → `public/` for static files served from `/`

CREATE-NEXT-APP FLAGS

FLAG	EFFECT
`--ts / --js`	TypeScript (default) or JavaScript
`--tailwind`	Tailwind CSS v4 (default)
`--eslint / --biome / --no-linter`	Linter choice
`--react-compiler`	Enable the React Compiler
`--src-dir`	Put `app/` inside `src/`
`--empty`	Minimal starter page
`--agents-md`	Add `AGENTS.md` / `CLAUDE.md` (default)
`--webpack`	Use webpack instead of Turbopack

MANUAL INSTALL

STEP	COMMAND / FILE
Install	`npm i next@latest react@latest react-dom@latest`
Types	`npm i -D typescript @types/react @types/node`
Scripts	`"dev": "next dev"` etc.
Root layout	`app/layout.tsx`
Home page	`app/page.tsx`

⚠ COMMON MISTAKES

- Running an old global `create-next-app`; always use `@latest` so you get 16.x templates
- Using `npm run dev -p 4000`: npm needs `--` before flags (`npm run dev -- -p 4000`)
- Setting `PORT` in `.env`; the server boots before `.env` loads, so use the shell env

✓ TIPS & BEST PRACTICES

- Pass every flag in CI so scaffolding never blocks on a prompt
- Keep `--src-dir` if you want config files separated from app code
- Use `npx @next/codemod@canary upgrade latest` to bring older apps forward

QUICK SUMMARY

- `npx create-next-app@latest` — scaffold a project
- `--yes` — accept recommended defaults
- `--api` — route-handler-only project
- `npm run dev` — dev server on port 3000

Project Structure & Special Files

HOW FILES BECOME ROUTES

Inside `app/`, **folders define URL segments** and **special files** define the UI for that segment. Only a `page` or `route` file makes a segment publicly reachable, so you can safely **colocate** components, tests, and styles next to the route that uses them.

TOP-LEVEL FILES

- `public/` — static assets at `/` (e.g. `/logo.svg`)
- `next.config.ts` — framework configuration
- `proxy.ts` — request interception (was `middleware.ts`)
- `instrumentation.ts` — server startup hooks

A TYPICAL TREE

```
my-app/
├─ app/
│  ├─ layout.tsx          # root layout (required)
│  ├─ page.tsx            # /
│  ├─ globals.css
│  ├─ not-found.tsx       # global 404 UI
│  ├─ dashboard/
│  │  ├─ layout.tsx       # wraps /dashboard/*
│  │  ├─ page.tsx         # /dashboard
│  │  ├─ loading.tsx      # Suspense fallback
│  │  ├─ error.tsx        # error boundary
│  │  └─ chart.tsx        # colocated, NOT a route
│  └─ api/health/route.ts # GET /api/health
├─ public/logo.svg        # /logo.svg
├─ proxy.ts
└─ next.config.ts
```

READING THE TREE

- → Folder = URL segment (`dashboard/` → `/dashboard`)
- → `chart.tsx` is private: no `page` export means no URL
- → `route.ts` cannot share a folder with `page.tsx`
- → Nesting order: layout → template → error → loading → not-found → page
- → `src/app` works the same; config files stay at the root

SPECIAL FILES

FILE	PURPOSE
`page`	Unique UI for a route; makes it public
`layout`	Shared UI that persists across navigation
`template`	Like layout but remounts on navigation
`loading`	Instant Suspense fallback
`error / global-error`	Error boundaries
`not-found`	UI for `notFound()` and unknown URLs
`default`	Fallback for parallel route slots
`route`	API endpoint (Route Handler)

FOLDER CONVENTIONS

PATTERN	MEANING
`[slug]`	Dynamic segment
`[...slug] / [[...slug]]`	Catch-all / optional
`(group)`	Route group; not in URL
`_folder`	Private; never routed
`@slot`	Parallel route slot
`(.)folder`	Intercepting route

⚠ COMMON MISTAKES

- Putting `page.tsx` and `route.ts` in the same folder; Next.js reports a conflict
- Expecting files in `public/` to be processed; they are served as-is with no hashing
- Naming a helper `page.tsx` deep in a folder and accidentally publishing a URL

✓ TIPS & BEST PRACTICES

- Prefix internal folders with `_` (e.g. `_components`) to make intent obvious
- Use `(group)` folders to organize by feature without changing URLs
- Keep shared code in `lib/` or `components/` outside `app/` if you prefer separation

QUICK SUMMARY

- `app/` — folders map to URL segments
- `page` / `route` — the only public entry points
- `loading` / `error` — automatic Suspense and error boundaries
- `public/` — static files served from `/`

next.config.ts

CONFIGURING THE FRAMEWORK

`next.config.ts` exports a typed `NextConfig` object (or a function receiving the build **phase**). It controls build output, image optimization, redirects and rewrites, response headers, and opt-in features like **Cache Components** and the **React Compiler**. Changes require restarting `next dev`.

16.X CONFIG CHANGES

- `experimental.dynamicIO` → top-level `cacheComponents`
- `experimental.turbopack` → top-level `turbopack`
- `reactCompiler` and `typedRoutes` are top-level (stable)
- `serverRuntimeConfig` / `publicRuntimeConfig` removed: use env vars

A REALISTIC CONFIG

```ts
import type { NextConfig } from 'next'

const nextConfig: NextConfig = {
  cacheComponents: true,          // 'use cache' + Partial Prerendering
  reactCompiler: true,            // needs babel-plugin-react-compiler
  typedRoutes: true,              // type-checked <Link href>
  images: { remotePatterns: [new URL('https://cdn.example.com/**')] },
  async redirects() {
    return [{ source: '/old-blog/:slug', destination: '/blog/:slug',
              permanent: true }]                    // 308
  },
  async rewrites() {
    return [{ source: '/docs/:path*',
              destination: 'https://docs.example.com/:path*' }]
  },
}
export default nextConfig
```

KEY OPTIONS

- → `cacheComponents` — enables `'use cache'`, `cacheLife`, PPR
- → `reactCompiler` — automatic memoization at build time
- → `redirects()` — `permanent: true` = 308, `false` = 307
- → `rewrites()` — proxy a path without changing the URL

COMMON OPTIONS

OPTION	PURPOSE
`output`	`'standalone'` or `'export'`
`basePath`	Serve the app under `/sub-path`
`headers()`	Add response headers per path
`serverExternalPackages`	Skip bundling native deps
`cacheLife`	Custom cache profiles
`partialPrefetching`	16.3 opt-in prefetch model

FUNCTION FORM

CODE	USE
`(phase) => ({...})`	Vary config per phase
`PHASE_DEVELOPMENT_SERVER`	`next dev`
`PHASE_PRODUCTION_BUILD`	`next build`
`PHASE_PRODUCTION_SERVER`	`next start` (all from `next/constants`)

⚠ COMMON MISTAKES

- Leaving `images.domains` in place: it is deprecated in 16; use `remotePatterns`
- Expecting `rewrites()` to change the browser URL; that is what `redirects()` does
- Editing config while `next dev` runs and wondering why nothing changed; restart it

✓ TIPS & BEST PRACTICES

- Type the export with `NextConfig` to catch typos in option names
- Use `permanent: false` while testing redirects; 308s are cached hard by browsers
- Keep secrets out of config: anything in `env:` is inlined into the client bundle

QUICK SUMMARY

- `NextConfig` — typed default export
- `cacheComponents` — opt into the new caching model
- `redirects()` / `rewrites()` / `headers()` — async arrays
- `output` — `'standalone'` or `'export'` builds

CLI & Dev Tooling

THE NEXT COMMAND

The `next` CLI runs every phase of the app. `next dev` gives hot reloading and the **DevTools** overlay, `next build` produces an optimized `.next/` folder, and `next start` serves it. In 16.x **`next lint` is removed**: run ESLint or Biome directly. Dev output goes to `.next/dev`, so `dev` and `build` can run at the same time.

16.X CHANGES

- Turbopack is default; opt out with `--webpack`
- `next lint` removed; `next build` no longer lints
- `next upgrade` and `next experimental-analyze` added in 16.1
- A lockfile blocks two `next dev` runs in one project

EVERYDAY COMMANDS

```
next dev                          # Turbopack dev server on :3000
next dev -p 4000 -H 0.0.0.0       # custom port, reachable on LAN
next dev --experimental-https     # self-signed HTTPS for OAuth/webhooks
next dev --inspect                # attach a Node debugger (16.1+)
next build                        # production build → .next/
next build --webpack              # opt out of Turbopack
next build --debug-prerender      # readable prerender errors
next start -p 8080                # serve the production build
next typegen && tsc --noEmit      # route types without a build
next info                         # versions for bug reports
next upgrade                      # bump Next.js (16.1+)
next experimental-analyze         # Turbopack bundle analyzer
```

NOTES

→ `-p` / `PORT` — port; `PORT` cannot come from `.env`

→ `--debug-prerender` — dev-only; never deploy that build

→ `typegen` — generates `PageProps`, `LayoutProps`, `next-env.d.ts`

→ `experimental-analyze --output` writes to `.next/diagnostics`

LINTING AFTER NEXT LINT

TOOL	SETUP
ESLint	`eslint-config-next` (flat config)
Run	`eslint .` in a `lint` script
Codemod	`npx @next/codemod@canary next-lint-to-eslint-cli .`
Biome	`biome check .`; Next/React domains built in

PACKAGE.JSON SCRIPTS

SCRIPT	COMMAND
dev	`next dev`
build	`next build`
start	`next start`
lint	`eslint .` or `biome check`
typecheck	`next typegen && tsc --noEmit`

⚠ COMMON MISTAKES

- Calling `next lint` in CI after upgrading; the command no longer exists in 16
- Assuming `next build` catches lint errors; add a separate lint step
- Running `tsc` without `next typegen` first; route helper types will be missing

✓ TIPS & BEST PRACTICES

- Use `next build --debug-build-paths="app/blog/**"` to rebuild only a few routes while debugging
- Add `.next/` and `next-env.d.ts` to `.gitignore`
- Turn telemetry off with `next telemetry --disable` if policy requires it

QUICK SUMMARY

- `next dev` — HMR dev server (Turbopack)
- `next build` / `next start` — production build and server
- `next typegen` — route types for CI
- `next lint` — removed; use ESLint or Biome

File-System Routing

FOLDERS ARE URLS

Every folder inside `app/` is a **route segment**, and the path of folders from `app/` down to a `page.tsx` is the URL. A route becomes public only when its folder contains a `page` file whose **default export** is a React component. The `page.tsx` directly inside `app/` is the **index route** `/`.

RULES

- Page files: `.tsx`, `.ts`, `.jsx`, `.js` (see `pageExtensions`)
- Pages can be `async` Server Components
- Segment names become URL text exactly (keep them lowercase)
- Unknown URLs render `app/not-found.tsx`

FOLDER → URL MAPPING

```
app/page.tsx                          →  /
app/about/page.tsx                    →  /about
app/blog/page.tsx                     →  /blog
app/blog/[slug]/page.tsx              →  /blog/hello-world
app/shop/[...path]/page.tsx           →  /shop/a/b/c
app/(marketing)/pricing/page.tsx      →  /pricing
app/_lib/db.ts                        →  (private, never routed)
app/dashboard/settings/page.tsx       →  /dashboard/settings
app/api/users/route.ts                →  /api/users  (endpoint)
```

READING THE MAP

- → Each `/` in the URL is one folder level
- → `[slug]` matches any single segment value
- → `(marketing)` organizes files without adding to the URL
- → `_lib` is ignored by the router entirely
- → `route.ts` returns a `Response`, not UI

A MINIMAL PAGE

```tsx
// app/about/page.tsx
import type { Metadata } from 'next'

export const metadata: Metadata = { title: 'About' }

export default function AboutPage() {
  return <h1>About us</h1>
}
```

SEGMENT TYPES

FOLDER	MATCHES
`about`	`/about` only
`[id]`	`/42`, `/abc`
`[...slug]`	`/a`, `/a/b` (not `/`)
`[[...slug]]`	`/`, `/a`, `/a/b`
`(group)`	Nothing; organizational
`_private`	Nothing; excluded

⚠ COMMON MISTAKES

- Using a named export (`export function Page`) instead of `export default`; the route 404s
- Creating `app/about.tsx` like the Pages Router; App Router needs `app/about/page.tsx`
- Mixed-case folder names (`/AboutUs`) that produce awkward, case-sensitive URLs

✓ TIPS & BEST PRACTICES

- Colocate components beside the page that uses them; only `page` is public
- Use `typedRoutes: true` so links to missing routes fail type-checking
- Check the `next build` route list to confirm every URL you expect exists

QUICK SUMMARY

- `page.tsx` — default export makes the URL public
- Folder nesting — URL path segments
- `app/page.tsx` — the index route `/`
- `not-found.tsx` — unmatched URLs

Dynamic Routes

SEGMENTS FILLED AT REQUEST TIME

Wrap a folder name in brackets to capture part of the URL. The captured values arrive in the `params` prop, which in Next.js 15+ is a **Promise** you must `await`. Export `generateStaticParams()` to **prerender** known values at build time; other values render on demand.

PARAMS SHAPES

- `[id]` → `{ id: string }`
- `[...slug]` → `{ slug: string[] }`
- `[[...slug]]` → `{ slug?: string[] }`
- Values arrive URL-decoded strings; parse numbers yourself

DYNAMIC PAGE WITH STATIC PARAMS

```tsx
// app/blog/[slug]/page.tsx
import { notFound } from 'next/navigation'
import { getPost, getAllSlugs } from '@/lib/posts'

export async function generateStaticParams() {
  const slugs = await getAllSlugs()           // ['intro', 'v16']
  return slugs.map((slug) => ({ slug }))      // prerendered at build
}

export default async function Post(props: PageProps<'/blog/[slug]'>) {
  const { slug } = await props.params         // must await
  const post = await getPost(slug)
  if (!post) notFound()                       // renders not-found.tsx
  return <article><h1>{post.title}</h1>{post.body}</article>
}
```

WHAT HAPPENS

→ `generateStaticParams` — returns one object per page to build

→ `PageProps<'/blog/[slug]'>` — typed `params` and `searchParams`

→ `await props.params` — sync access was removed in 16

→ `notFound()` — throws; no `return` needed

→ Unlisted slugs still work and are rendered on demand

CATCH-ALL EXAMPLES

ROUTE	URL	PARAMS
`[slug]`	/blog/a	`{ slug: 'a' }`
`[...slug]`	/docs/a/b	`{ slug: ['a','b'] }`
`[...slug]`	/docs	404 (no match)
`[[...slug]]`	/docs	`{ slug: undefined }`
`[cat]/[id]`	/shoes/42	`{ cat: 'shoes', id: '42' }`

STATIC PARAMS OPTIONS

EXPORT	EFFECT
`generateStaticParams`	List params to prerender
`return []`	Prerender none; render all on demand
`dynamicParams = false`	Unlisted params 404 (pre-Cache Components)
`Parent params`	Child receives `{ params }` argument
`Cache Components`	Unlisted pages get an instant shell (16.3)

⚠ COMMON MISTAKES

- Destructuring `params` synchronously (`{ params: { slug } }`); in 16 it is a Promise
- Forgetting that `[...slug]` does not match the bare parent URL; use `[[...slug]]`
- Returning raw strings from `generateStaticParams` ; it must return objects keyed by segment name

✓ TIPS & BEST PRACTICES

- Validate params with Zod or a lookup before querying; they are user input
- Use `PageProps<'/route'>` instead of hand-writing `{ params: Promise<...> }`
- Prerender your most-visited slugs and let the long tail render on demand

QUICK SUMMARY

- `[slug]` — one dynamic segment
- `[...slug]` / `[[...slug]]` — catch-all / optional
- `await params` — params is a Promise
- `generateStaticParams` — prerender known values

Layouts & Templates

SHARED UI THAT PERSISTS

A `layout.tsx` wraps every page and nested layout below it via the `children` prop. Layouts **persist across navigation**: they don't remount, so state and scroll position survive. The **root layout** is required and must render `<html>` and `<body>`. A `template.tsx` looks the same but **remounts** on every navigation.

LAYOUT FACTS

- Layouts nest: root → `dashboard/layout` → page
- Layouts receive `params` (Promise) but not `searchParams`
- A layout does not re-render when only the child page changes
- Export `metadata` from layouts for shared titles

ROOT + NESTED LAYOUT

```tsx
// app/layout.tsx
import './globals.css'
export default function RootLayout({ children }: LayoutProps<'/'>) {
  return (
    <html lang="en"><body><Header />{children}</body></html>
  )
}

// app/dashboard/layout.tsx
export default async function DashLayout(
  { children }: LayoutProps<'/dashboard'>) {
  const user = await getCurrentUser()
  return (
    <div className="grid grid-cols-[240px_1fr]">
      <Sidebar user={user} />
      <main>{children}</main>
    </div>
  )
}
```

NOTES

- → `globals.css` — import global CSS once, in the root layout
- → `LayoutProps<'/dashboard'>` — types `children`, `params`, slots
- → `DashLayout` — wraps every `/dashboard/*` page
- → `Sidebar` keeps its state as the user moves between pages
- → Async layouts can fetch data like pages

LAYOUT VS TEMPLATE

BEHAVIOR	LAYOUT	TEMPLATE
Remounts on navigation	✖	✔
Keeps client state	✔	✖
Re-runs effects	✖	✔
Required at root	✔	✖
Typical use	Shells, nav	Enter animations, logging

ROOT LAYOUT RULES

RULE	DETAIL
`<html> + <body>`	Must be rendered by the root layout
No `<head>` tag	Use the Metadata API instead
Multiple roots	Allowed via route groups
Cross-root nav	Triggers a full page load
searchParams	Not available; read in page or client hook

⚠ COMMON MISTAKES

- Reading `searchParams` in a layout; layouts don't re-render when the query changes, so the value goes stale
- Adding a `<head>` tag manually; use `metadata` or `generateMetadata`
- Using a layout for per-page auth checks; a layout doesn't re-run on every child navigation

✓ TIPS & BEST PRACTICES

- Push data fetching down to the component that needs it so layouts stay fast
- Use `template.tsx` when an animation or `useEffect` must run on every page change
- Pass server data to client widgets in the layout as serializable props

QUICK SUMMARY

- `layout.tsx` — persistent shared UI via `children`
- Root layout — required; renders `<html>` and `<body>`
- `template.tsx` — remounts per navigation
- `LayoutProps<'/path'>` — typed layout props

Linking & Navigation

MOVING BETWEEN ROUTES

Use `<Link>` for navigation: it renders an `<a>`, **prefetches** routes as they enter the viewport, and performs a client-side transition without a full reload. Use `useRouter()` for programmatic navigation in Client Components, and `redirect()` on the server (Server Components, Server Actions, Route Handlers).

PREFETCH BEHAVIOR

- Prefetching runs only in production builds
- Default: prefetch the shared shell / `loading` boundary
- `prefetch={true}` : prefetch the full page (per-link data in 16.3)
- `prefetch={false}` : no prefetch until hover

LINKS, ROUTER, AND REDIRECTS

```tsx
import Link from 'next/link'
<Link href="/blog/hello">Read</Link>
<Link href={{ pathname: '/search', query: { q: 'next' } }}>Search</Link>
<Link href="/dashboard" prefetch={false} replace scroll={false}>Go</Link>

'use client'
import { useRouter, usePathname, useSearchParams } from 'next/navigation'
const router = useRouter()
router.push('/cart')            // new history entry
router.replace('/login')        // replace current entry
router.back(); router.refresh() // refresh re-fetches server data
const path = usePathname()      // '/blog/hello'
const q = useSearchParams().get('q')   // 'next'

import { redirect, permanentRedirect } from 'next/navigation'
redirect('/login')              // 307 (303 in Server Actions)
```

NOTES

- → Import hooks from `next/navigation`, not `next/router`
- → `replace` — swaps the history entry instead of pushing
- → `scroll={false}` — keep scroll position after navigating
- → `router.refresh()` — re-renders Server Components, keeps client state
- → `permanentRedirect()` — 308 for moved content

<LINK> PROPS

PROP	PURPOSE
`href`	String, URL object, or typed Route
`prefetch`	`null` (auto) · `true` · `false`
`replace`	Replace history entry
`scroll`	Scroll to top (default `true`)
`onNavigate`	Run code or `preventDefault()`
`transitionTypes`	Tags for `<ViewTransition>`

NAVIGATION HOOKS

HOOK	RETURNS
`useRouter()`	push, replace, back, refresh, prefetch
`usePathname()`	Current path string
`useSearchParams()`	Read-only `URLSearchParams`
`useParams()`	Dynamic params object
`useLinkStatus()`	`{ pending }` inside a `<Link>`
`useSelectedLayoutSegment()`	Active child segment

⚠ COMMON MISTAKES

- Calling `redirect()` inside `try/catch`; it works by throwing, so the catch swallows it
- Using plain `<a>` tags for internal links; you lose prefetching and do a full reload
- Reading `useSearchParams()` without a `<Suspense>` boundary; the route can't be prerendered

✓ TIPS & BEST PRACTICES

- Show a spinner per link with `useLinkStatus()` inside a child of `<Link>`
- Use `onNavigate` to close menus the instant a link is clicked
- Prefer `<Link>` over `router.push` whenever the target is known while rendering

QUICK SUMMARY

- `<Link>` — client navigation with prefetching
- `useRouter()` — programmatic navigation (client)
- `redirect()` — server-side 307/303 redirect
- `usePathname` / `useSearchParams` — read the URL

Loading, Error & Not Found UI

BUILT-IN BOUNDARIES

Special files turn into React boundaries automatically. `loading.tsx` becomes a **Suspense** fallback shown instantly while the segment streams. `error.tsx` becomes an **error boundary** (a Client Component) that catches render errors below it. `not-found.tsx` renders when you call `notFound()` or no route matches.

16.X UPDATES

- `error.tsx` gets a `retry()` prop (stable in 16.3)
- `retry()` re-fetches; old `reset()` only re-renders
- `catchError` from `next/error` builds custom boundaries
- `forbidden()` / `unauthorized()` need `experimental.authInterrupts`

THE THREE FILES

```tsx
// app/dashboard/loading.tsx
export default function Loading() { return <p>Loading dashboard…</p> }

// app/dashboard/error.tsx
'use client'                                  // must be a Client Component
export default function Error({ error, retry }: {
  error: Error & { digest?: string }; retry: () => void }) {
  return (
    <div role="alert">
      <p>Couldn't load data ({error.digest})</p>
      <button onClick={() => retry()}>Try again</button>
    </div>
  )
}

// app/blog/[slug]/not-found.tsx
export default function NotFound() { return <h2>Post not found</h2> }
```

NOTES

- → `loading.tsx` wraps `page.tsx` in `<Suspense>` for you
- → `error.digest` — hash that matches the server log entry
- → In production, server error messages are replaced with a generic one
- → `error.tsx` doesn't catch errors in its own segment's `layout.tsx`
- → `global-error.tsx` must render its own `<html>` and `<body>`

THROWING HELPERS

CALL	RENDERS	STATUS
`notFound()`	nearest `not-found.tsx`	404
`forbidden()`	`forbidden.tsx`	403
`unauthorized()`	`unauthorized.tsx`	401
`redirect(url)`	target route	307
`throw new Error()`	nearest `error.tsx`	500

WHICH FILE CATCHES IT

ERROR IN	CAUGHT BY
`page / child layout`	`error.tsx` in same folder
`layout of segment`	parent's `error.tsx`
`root layout`	`global-error.tsx`
`event handler`	Not caught; use try/catch
`Server Action`	Return errors as values

⚠ COMMON MISTAKES

- Leaving `'use client'` off `error.tsx`; error boundaries must be Client Components
- Wrapping `notFound()` or `redirect()` in try/catch; use `unstable_rethrow(err)` in the catch
- Expecting a 404 status after streaming starts; once the shell is sent the status is already 200

✓ TIPS & BEST PRACTICES

- Prefer `retry()` over `reset()` so Server Components re-fetch on recovery
- Log `error.digest` to correlate client reports with server logs
- Put `loading.tsx` low in the tree so more of the page shows immediately

QUICK SUMMARY

- `loading.tsx` — automatic Suspense fallback
- `error.tsx` — client error boundary with `retry()`
- `not-found.tsx` + `notFound()` — 404 UI
- `global-error.tsx` — root layout failures

Sheet 11

Route Groups & Private Folders

(group) · multiple root layouts · _folder · opt out of layouts · feature folders

ORGANIZING WITHOUT CHANGING URLS

A folder wrapped in parentheses, `(marketing)`, is a **route group**: it organizes files and can hold its own `layout.tsx`, but its name never appears in the URL. A folder prefixed with an underscore, `_components`, is **private**: the router ignores it and everything inside it.

USES

- Give sections different layouts (`(shop)` vs `(admin)`)
- Create **multiple root layouts** with separate `<html>`
- Keep a page out of a shared layout
- Group by team or feature

TWO SECTIONS, TWO LAYOUTS

```
app/
├─ (marketing)/
│  ├─ layout.tsx          # header + footer
│  ├─ page.tsx            # /
│  └─ pricing/page.tsx    # /pricing
├─ (app)/
│  ├─ layout.tsx          # sidebar shell, auth check
│  ├─ dashboard/page.tsx  # /dashboard
│  └─ settings/page.tsx   # /settings
├─ (auth)/login/page.tsx  # /login — no header, no sidebar
├─ _components/Button.tsx # private, never routed
└─ _lib/db.ts             # private
```

READING IT

- → No root `app/layout.tsx` here: each group's layout is a root layout
- → Each root layout must render `<html>` and `<body>`
- → `/login` skips both shells because `(auth)` has no layout
- → Navigating between root layouts causes a full page load
- → `_components` keeps shared UI next to routes safely

NAMING CONVENTIONS

FOLDER	IN URL?	ROUTABLE?
`(group)`	✖	✔ (its children)
`_private`	✖	✖
`%5Fname`	✔ as `_name`	✔
`normal`	✔	✔

CONFLICTS TO AVOID

MISTAKE	RESULT
`(a)/about + (b)/about`	Build error: same URL
`Two groups both with page.tsx at root`	Build error for /
`Root layout missing <html>`	Runtime error
`Layout in group above page`	Applies to that group only

⚠ COMMON MISTAKES

- Resolving two pages to the same URL from different groups; Next.js fails the build
- Forgetting `<html>` / `<body>` in each layout once the top-level root layout is removed
- Expecting `_folder` to work as a URL; use `%5Ffolder` if you really need a leading underscore

✓ TIPS & BEST PRACTICES

- Use `(app)` for authenticated areas so the auth-aware shell lives in one layout
- Name groups by audience or feature, not by implementation detail
- Private folders are optional: only `page` and `route` are ever public anyway

QUICK SUMMARY

- `(group)` — organize; not part of the URL
- Group layouts — different shells per section
- Multiple root layouts — full reload between them
- `_folder` — opt out of routing

Parallel & Intercepting Routes

SEVERAL PAGES IN ONE LAYOUT

Parallel routes render multiple pages in the same layout at once. Each `@slot` folder is passed to the parent layout as a prop. **Intercepting routes** load a route inside the current layout on client navigation (for example, a photo opens in a modal) while a direct visit still renders the full page.

RULES

- Slots are not URL segments: `@team/members` → `/members`
- Every slot needs a `default.tsx` (build fails without it in 16)
- `default.tsx` is shown when a slot has no match after reload
- Interception happens only on soft navigation

PHOTO MODAL PATTERN

```
app/
├─ layout.tsx                 # receives { children, modal }
├─ @modal/
│  ├─ default.tsx             # export default () => null
│  └─ (.)photo/[id]/page.tsx  # modal version (intercepted)
├─ photo/[id]/page.tsx        # full page on direct visit
└─ page.tsx                   # gallery with <Link href="/photo/1">

// app/layout.tsx
export default function Layout({ children, modal }: LayoutProps<'/'>) {
  return <html><body>{children}{modal}</body></html>
}
```

HOW IT PLAYS OUT

- → Click `/photo/1` in the gallery → `@modal` renders the modal
- → Refresh or share the link → full `photo/[id]/page.tsx`
- → `(.)` intercepts a sibling segment at the same level
- → Close with `router.back()` from the modal
- → `modal` prop name comes from the `@modal` folder

INTERCEPTION PREFIXES

PREFIX	MATCHES FROM
`(.)`	Same level
`(..)`	One level up
`(..)(..)`	Two levels up
`(...)`	The app root

COMMON USES

PATTERN	HOW
Dashboards	`@analytics` + `@team` slots
Role-based UI	Return `admin` or `user` slot
Tabs per slot	Nested pages inside a slot
Modals	`@modal` + `(.)` intercept
Close modal	`router.back()` or catch-all `null`

⚠ COMMON MISTAKES

- Omitting `default.tsx` for a slot; Next.js 16 fails the build instead of guessing
- Counting `(..)` by folders on disk; it follows route segments, so groups and slots don't count
- Expecting interception on a hard refresh; direct loads always render the real page

✓ TIPS & BEST PRACTICES

- Return `null` from `@modal/default.tsx` so the modal is hidden by default
- Add `@modal/[...catchAll]/page.tsx` returning `null` to close modals on navigation
- Use slots to give each dashboard panel its own `loading.tsx` and `error.tsx`

QUICK SUMMARY

- `@slot` — parallel route passed as a layout prop
- `default.tsx` — required fallback per slot
- `(.)`, `(..)`, `(...)` — interception levels
- Modal = `@modal` slot + intercepted route

Server Components

default in app/ · async components · direct DB access · server-only · no hooks

RENDERING ON THE SERVER

Every component in `app/` is a **React Server Component (RSC)** unless marked otherwise. Server Components run only on the server (at build time or per request), can be `async`, and can read databases, files, and secrets directly. Their code is **never sent to the browser**, which keeps bundles small.

WHAT THEY CAN'T DO

- No `useState`, `useEffect`, or other client hooks
- No event handlers (`onClick`, `onChange`)
- No browser APIs (`window`, `localStorage`)
- Props passed to Client Components must be serializable

QUERY THE DATABASE IN A COMPONENT

```tsx
// app/orders/page.tsx  — a Server Component (no directive)
import 'server-only'                      // build error if imported by client
import { db } from '@/lib/db'
import { OrderFilter } from './order-filter' // a Client Component

export default async function OrdersPage() {
  const orders = await db.order.findMany({
    orderBy: { createdAt: 'desc' }, take: 20,
  })
  const apiKey = process.env.STRIPE_SECRET_KEY  // safe: never shipped
  return (
    <section>
      <OrderFilter />                       {/* interactive island */}
      <ul>{orders.map((o) => <li key={o.id}>{o.total}</li>)}</ul>
    </section>
  )
}
```

NOTES

- → No directive needed: `app/` defaults to server
- → `import 'server-only'` guards modules holding secrets or DB code
- → `await` directly in the component body
- → Secrets in non-`NEXT_PUBLIC_` env vars stay on the server
- → Client Components can be rendered as children of server ones

SERVER VS CLIENT: WHEN TO USE

NEED	SERVER	CLIENT
Fetch data / query DB	✔	✖
Use secrets / API keys	✔	✖
Large dependencies (markdown, date libs)	✔	✖
State, effects, event handlers	✖	✔
Browser APIs	✖	✔
Custom hooks / context consumers	✖	✔

WHAT THE SERVER SENDS

OUTPUT	USED FOR
HTML	Fast first paint on page load
RSC payload	Tree + client props for hydration
Client JS	Only for `'use client'` modules
Streamed chunks	Content inside <Suspense>

⚠ COMMON MISTAKES

- Adding hooks to a Server Component and getting a build error; split the interactive part into a client file
- Passing functions or class instances as props to Client Components; only serializable values cross the boundary
- Importing a DB module from a Client Component; add `server-only` so the mistake fails at build time

✓ TIPS & BEST PRACTICES

- Keep pages and layouts as Server Components; push `'use client'` to the leaves
- Heavy libraries used only for rendering (syntax highlighters, markdown) belong on the server
- Use `React.cache()` to dedupe the same query across components in one request

QUICK SUMMARY

- Default in `app/` — Server Component
- `async` components — fetch with `await`
- `server-only` — guard secret-holding modules
- No hooks or event handlers on the server

Client Components

OPTING INTO THE BROWSER

Add `'use client'` at the top of a file to mark it as a **client boundary**. That module and everything it imports become part of the client bundle: they are prerendered to HTML on the server, then **hydrated** in the browser so state, effects, and event handlers work. You only need the directive at the entry point of an interactive subtree.

BOUNDARY RULES

- Directive must be the first statement in the file
- Imports of a client file are client code too
- Props from server → client must be serializable
- Server Actions are the one function you may pass

AN INTERACTIVE ISLAND

```tsx
// app/products/add-to-cart.tsx
'use client'
import { useState, useTransition } from 'react'
import { addToCart } from './actions'          // a Server Action

export function AddToCart({ productId }: { productId: string }) {
  const [qty, setQty] = useState(1)
  const [pending, startTransition] = useTransition()
  return (
    <div>
      <input type="number" value={qty} min={1}
        onChange={(e) => setQty(Number(e.target.value))} />
      <button disabled={pending}
        onClick={() => startTransition(() => addToCart(productId, qty))}>
        {pending ? 'Adding…' : 'Add to cart'}
      </button>
    </div>
  )
}
```

NOTES

→ `'use client'` — this file is the boundary

→ `productId` — a string prop from the Server Component parent

→ `addToCart` — Server Actions can be imported into client code

→ `useTransition` — keeps the UI responsive during the action

→ Still SSR'd to HTML; becomes interactive after hydration

SERIALIZABLE PROPS

TYPE	ALLOWED
`string, number, boolean, null`	✔
`Plain objects / arrays`	✔
`Date, Map, Set, BigInt`	✔
`Promise (resolve with use())`	✔
`JSX / Server Components as children`	✔
`Server Actions`	✔
`Functions, class instances`	✖

BROWSER-ONLY CODE

NEED	APPROACH
`window / localStorage`	Read inside `useEffect`
`Library touching window`	`next/dynamic` with `ssr: false`
`Guard a module`	`import 'client-only'`
`Hydration mismatch`	Avoid `Date.now()` in render

⚠ COMMON MISTAKES

- Accessing `window` during render; Client Components still prerender on the server, so it crashes
- Marking a high-level layout `'use client'`; the whole subtree and its imports ship to the browser
- Passing an inline arrow function from a Server Component as a prop; functions aren't serializable

✓ TIPS & BEST PRACTICES

- Pass server-rendered content into Client Components as `children` to keep it on the server
- Use `suppressHydrationWarning` only for intentional mismatches like timestamps
- Name client files clearly (`*-client.tsx` or a `client/` folder) so reviewers spot boundaries

QUICK SUMMARY

- `'use client'` — marks a client boundary
- Hooks, state, events — client only
- Props — must be serializable
- `client-only` — guard browser-only modules

Composition Patterns

MIXING SERVER AND CLIENT

A Client Component can't import a Server Component, but it **can render one passed as a prop** (usually `children`). That rule drives the main patterns: keep data fetching in Server Components, wrap only the interactive parts in client boundaries, and pass server-rendered output through slots.

GOLDEN RULES

- Server → Client: pass data or JSX as props
- Client → Server: only via `children` or other slots
- Context lives in a client provider component
- Start promises on the server, read them with `use()`

PROVIDERS AND SLOTS

```tsx
// app/providers.tsx
'use client'
import { ThemeProvider } from 'next-themes'
export function Providers({ children }: { children: React.ReactNode }) {
  return <ThemeProvider attribute="class">{children}</ThemeProvider>
}

// app/layout.tsx  (Server Component)
import { Providers } from './providers'
export default function RootLayout({ children }: LayoutProps<'/'>) {
  return (
    <html lang="en" suppressHydrationWarning>
      <body><Providers>{children}</Providers></body>
    </html>
  )
}
```

WHY IT WORKS

- → `Providers` is the only client file; pages stay server-rendered
- → `children` is rendered on the server and passed in as a slot
- → Wrap third-party providers that use context in your own client file
- → Render the provider as deep as possible in the tree

PASS A PROMISE, READ WITH USE()

```tsx
// page.tsx (server) — don't await, pass the promise
const reviews = getReviews(id)
return <Suspense fallback={<Spinner />}>
  <Reviews data={reviews} /></Suspense>

// reviews.tsx (client)
'use client'
import { use } from 'react'
export function Reviews({ data }: { data: Promise<Review[]> }) {
  const reviews = use(data)    // suspends until resolved
  return <ul>{reviews.map(r => <li key={r.id}>{r.text}</li>)}</ul>
}
```

PATTERN CHEAT SHEET

PROBLEM	FIX
`Big layout needs one button`	Extract button to client file
`Client modal shows server data`	Pass it as `children`
`Library lacks 'use client'`	Re-export from a client file
`Need context everywhere`	Client `Providers` in layout

⚠ COMMON MISTAKES

- Importing a Server Component inside a `'use client'` file; it silently becomes a Client Component
- Creating a promise inside a Client Component render and passing it to `use()`; it re-creates every render
- Wrapping the whole `<body>` in many providers when only one section needs them

✓ TIPS & BEST PRACTICES

- Write a thin client wrapper for any third-party component that uses hooks but lacks the directive
- Use `server-only` in data modules so a wrong import fails fast
- Prefer slots (`children` , `sidebar`) over moving data fetching into the client

QUICK SUMMARY

- Slots — render server output inside client UI
- `Providers` — one client file for context
- Move `'use client'` to the leaves
- `use(promise)` — stream server data to client

React 19 Features Used in Next

use() · <Suspense> · useActionState · useOptimistic · <ViewTransition> · <Activity> · React Compiler

THE REACT LAYER UNDER NEXT.JS

The App Router ships a React **canary** build that includes everything in React 19.2. Several of these APIs are central to Next.js: `<Suspense>` drives streaming, `use()` reads promises, the form hooks power Server Actions, and `<Activity>` lets Cache Components keep previously visited pages alive in the background.

WHERE NEXT USES THEM

- `loading.tsx` = automatic `<Suspense>`
- Navigations are transitions, so they trigger `<ViewTransition>`
- With Cache Components, up to 3 routes stay mounted in `<Activity>`
- `reactCompiler: true` memoizes without `useMemo`

FORM HOOKS + OPTIMISTIC UI

```tsx
'use client'
import { useActionState, useOptimistic } from 'react'
import { sendMessage } from './actions'

export function Chat({ messages }: { messages: Msg[] }) {
  const [list, addOptimistic] = useOptimistic(
    messages, (cur, text: string) => [...cur, { id: 'tmp', text }])
  const [state, action, pending] = useActionState(
    async (prev: State, form: FormData) => {
      addOptimistic(String(form.get('text')))
      return sendMessage(prev, form)            // Server Action
    }, { error: null })
  return <form action={action}>
    {list.map((m) => <p key={m.id}>{m.text}</p>)}
    <input name="text" /><button disabled={pending}>Send</button>
    {state.error && <p role="alert">{state.error}</p>}
  </form>
}
```

NOTES

- → `useOptimistic` — show the message before the server confirms
- → `useActionState` — returns `[state, action, pending]`
- → Optimistic item is replaced when real data re-renders
- → Works without JS when the action is passed to `<form action>`

REACT 19.X APIS

API	PURPOSE
`use(promise | context)`	Read a value; suspends on promises
`<Suspense fallback>`	Streaming boundary
`useFormStatus()`	Pending state of the parent `<form>`
`useEffectEvent()`	Non-reactive logic in effects (19.2)
`<Activity mode>`	Hide UI but keep state (19.2)
`<ViewTransition>`	Animate navigations (canary)

REACT COMPILER

STEP	DETAIL
Install	`npm i -D babel-plugin-react-compiler`
Enable	`reactCompiler: true` in config
Opt out	`'use no memo'` directive
16.3	`experimental.turbopackRustReactCompiler`

⚠ COMMON MISTAKES

- Assuming state resets on navigation with Cache Components; `<Activity>` keeps it, so reset menus in a `useLayoutEffect` cleanup
- Calling `use()` on a promise created during the same client render; create it on the server or cache it
- Expecting `<ViewTransition>` to animate plain `setState` calls; it reacts to transitions and Suspense

✓ TIPS & BEST PRACTICES

- Import `ViewTransition` from `react`; no Next.js config is required
- Tag links with `transitionTypes={['nav-forward']}` for directional animations
- Turn on the React Compiler gradually and watch build times

QUICK SUMMARY

- `use()` — unwrap promises and context
- `useActionState` / `useOptimistic` — form state
- `<Activity>` — hidden but alive UI
- `reactCompiler` — automatic memoization

Fetching in Server Components

DATA WHERE YOU RENDER

In the App Router you fetch data by **awaiting it inside an async Server Component**: `fetch()`, an ORM, or any Node.js library. No special API is required. The main performance concern is avoiding **waterfalls**: start independent requests together, and deduplicate repeated calls in one request with `React.cache()`.

DEFAULTS (16.X)

- `fetch` is **not cached** by default
- Identical `fetch` GETs in one render are memoized
- With Cache Components, uncached data needs `<Suspense>` or `'use cache'`
- Errors thrown here go to the nearest `error.tsx`

PARALLEL REQUESTS + DEDUPED HELPER

```tsx
// lib/data.ts
import 'server-only'
import { cache } from 'react'
export const getUser = cache(async (id: string) =>
  db.user.findUnique({ where: { id } }))      // once per request

// app/users/[id]/page.tsx
export default async function Profile(props: PageProps<'/users/[id]'>) {
  const { id } = await props.params
  const [user, posts] = await Promise.all([      // start together
    getUser(id),
    fetch(`https://api.example.com/users/${id}/posts`)
      .then((r) => r.json()),
  ])
  return <UserView user={user} posts={posts} />
}
```

NOTES

- → `cache()` — memoizes per server request, not across users
- → `Promise.all` — total time = slowest request, not the sum
- → Use `Promise.allSettled` when one failure shouldn't fail the page
- → Any component calling `getUser(id)` reuses the same result

FETCH PATTERNS

PATTERN	CODE	WHEN
Sequential	`await a(); await b(a)`	b needs a's result
Parallel	`Promise.all([a(), b()])`	Independent data
Preload	`void getUser(id)`	Start early, await later
Streamed	`<Suspense><Slow/></Suspense>`	Don't block the page
Deduped	`cache(fn)`	Same data, many components

FETCH() OPTIONS

OPTION	EFFECT
`cache: 'no-store'`	Always fresh (default behavior)
`cache: 'force-cache'`	Data Cache (previous model)
`next: { revalidate: 60 }`	Time-based (previous model)
`next: { tags: ['posts'] }`	Tag for `revalidateTag`
`signal`	Abort / timeout

⚠ COMMON MISTAKES

- Chaining `await` calls that don't depend on each other; each one waits for the last
- Calling your own Route Handler from a Server Component; call the data function directly instead
- Expecting `React.cache()` to persist between requests; use `'use cache'` for that

✓ TIPS & BEST PRACTICES

- Wrap every data-access function in `cache()` once, in `lib/`, and reuse it everywhere
- Kick off slow requests with a preload call before awaiting faster ones
- Use `AbortSignal.timeout(5000)` so a slow API can't hang the render

QUICK SUMMARY

- `await` in async Server Components
- `Promise.all` — avoid waterfalls
- `cache()` — per-request dedupe
- `fetch` — uncached by default in 16

Streaming & Suspense

SEND WHAT'S READY FIRST

Streaming sends the page in chunks: the shell renders immediately, and each <Suspense> boundary is filled in as its data resolves. `loading.tsx` wraps a whole page in one boundary; explicit <Suspense> lets you stream individual sections so a slow widget never blocks the rest of the page.

BEHAVIOR

- Boundaries resolve independently, in any order
- HTML streams on first load; RSC payload on navigation
- Status code is sent before streaming (always 200)
- Search engines receive the fully streamed HTML

INDEPENDENT SECTIONS

```tsx
import { Suspense } from 'react'

export default function Dashboard() {
  return (
    <>
      <h1>Dashboard</h1>                        {/* in the shell */}
      <Suspense fallback={<CardSkeleton />}>
        <Revenue />                             {/* 300 ms */}
      </Suspense>
      <Suspense fallback={<TableSkeleton />}>
        <LatestOrders />                        {/* 2 s */}
      </Suspense>
    </>
  )
}

async function Revenue() {
  const data = await getRevenue()               // awaited inside boundary
  return <Chart data={data} />
}
```

READING IT

- → The heading and both skeletons render instantly
- → `Revenue` replaces its skeleton at ~300 ms
- → `LatestOrders` streams in at ~2 s without blocking Revenue
- → Awaiting in the page itself would block everything
- → Size skeletons to match content to avoid layout shift

LOADING.TSX VS <SUSPENSE>

ASPECT	LOADING.TSX	<SUSPENSE>
Scope	Whole segment	Any subtree
Setup	Drop in a file	Wrap components
Granularity	One fallback	Many fallbacks
Shows on nav	✔ (prefetched)	✔
Best for	Simple pages	Dashboards, feeds

AVOIDING WATERFALLS

PROBLEM	FIX
Parent awaits, then child fetches	Fetch in parallel or pass promise
Nested Suspense each fetch in turn	Start all requests at the top
One slow query blocks page	Isolate it in its own boundary
Boundary too high	Move <Suspense> closer to the data

⚠ COMMON MISTAKES

- Awaiting slow data in the page component above the boundary; the Suspense fallback never shows
- Putting `<Suspense>` inside the async component it should wrap; it must be a parent
- Trying to set a 404 after streaming starts; check existence before the first boundary

✓ TIPS & BEST PRACTICES

- Use the React DevTools Suspense panel to toggle boundaries and inspect fallbacks
- Pass a `key` to `<Suspense>` to re-show the fallback when params change
- Group related widgets in one boundary to avoid a 'popcorn' layout

QUICK SUMMARY

- Streaming — shell first, holes fill in later
- `loading.tsx` — page-level fallback
- `<Suspense>` — section-level fallback
- Await inside the boundary, not above it

Client-Side Fetching

WHEN THE BROWSER SHOULD FETCH

Most data should be fetched on the server. Fetch on the **client** when data depends on user interaction after load (search-as-you-type, infinite scroll), needs **polling** or real-time updates, or is per-user and not worth server rendering. Use a data library such as **SWR** or **TanStack Query** for caching, deduping, and revalidation.

GOOD CLIENT CASES

- Live data: notifications, stock prices
- Infinite scroll and pagination UI
- Optimistic updates with local cache
- Data needed only after a user action

SWR SEEDED FROM THE SERVER

```tsx
// app/inbox/page.tsx (server) — initial data, no spinner
const initial = await getMessages()
return <Inbox fallback={initial} />

// app/inbox/inbox.tsx
'use client'
import useSWR from 'swr'
const fetcher = (url: string) => fetch(url).then((r) => r.json())

export function Inbox({ fallback }: { fallback: Message[] }) {
  const { data, error, isLoading, mutate } = useSWR(
    '/api/messages', fetcher,
    { fallbackData: fallback, refreshInterval: 10_000 })   // poll 10 s
  if (error) return <p>Failed to load</p>
  return <MessageList items={data} onRead={() => mutate()} />
}
```

NOTES

- → `fallbackData` — first render uses server data
- → `/api/messages` — a Route Handler (`app/api/messages/route.ts`)
- → `refreshInterval` — background polling
- → `mutate()` — revalidate after a change
- → Same key in two components = one request

LIBRARY COMPARISON

FEATURE	SWR	TANSTACK QUERY
Hook	`useSWR(key, fn)`	`useQuery(opts)`
Mutate	`mutate()`	`useMutation()`
Devtools	✖	✔
SSR hydration	fallbackData	HydrationBoundary
Size	Small	Larger, more features

USEEFFECT PITFALLS

ISSUE	FIX
Race conditions	Use a library or AbortController
Double fetch in dev	StrictMode; libraries dedupe
No caching	SWR / TanStack Query
Waterfall after hydration	Fetch on server instead

⚠ COMMON MISTAKES

- Fetching first-render data in `useEffect`; users see a spinner the server could have avoided
- Calling a Route Handler with a relative URL from a Server Component; there is no browser origin on the server
- Putting `QueryClientProvider` in a Server Component; it must live in a client provider

✓ TIPS & BEST PRACTICES

- Seed client caches with server data (`fallbackData` or `HydrationBoundary`) to skip loading states
- Prefer Server Actions over custom POST endpoints for mutations from your own UI
- Create the TanStack `QueryClient` with `useState(() => new QueryClient())` so it's stable

QUICK SUMMARY

- Server first — client only for interactive data
- `useSWR` / `useQuery` — cache, dedupe, revalidate
- Route Handlers — JSON endpoints for the client
- Seed client caches from server data

params & searchParams

READING THE URL ON SERVER AND CLIENT

Pages receive `params` (dynamic segments) and `searchParams` (query string) as **Promises**. Layouts receive only `params`. Next.js generates global helpers (`PageProps`, `LayoutProps`, `RouteContext`) so these props are typed from your actual routes. In Client Components, read the same values with hooks.

KEY FACTS

- Reading `searchParams` makes the page render per request
- Values are `string | string[] | undefined`
- Types come from `next dev`, `next build`, or `next typegen`
- 16.3: `next/root-params` reads root params anywhere

SEARCH PAGE WITH FILTERS

```tsx
// app/shop/[category]/page.tsx
export default async function Shop(props: PageProps<'/shop/[category]'>) {
  const { category } = await props.params                  // 'shoes'
  const { sort = 'new', page = '1' } = await props.searchParams
  const items = await getItems({
    category, sort: String(sort), page: Number(page) })    // ?sort=price&page=2
  return <ItemGrid items={items} />
}

// components/sort-select.tsx (client)
'use client'
import { useParams, useSearchParams, useRouter } from 'next/navigation'
export function SortSelect() {
  const { category } = useParams<{ category: string }>()
  const sp = useSearchParams(); const router = useRouter()
  const set = (v: string) => { const n = new URLSearchParams(sp)
    n.set('sort', v); router.replace(`?${n}`) }
  return <select defaultValue={sp.get('sort') ?? 'new'}
    onChange={(e) => set(e.target.value)}>{/* … */}</select>
}
```

NOTES

- → `PageProps<'/shop/[category]'>` — no imports needed
- → Default values while destructuring the awaited object
- → Cast query values: they can be arrays (`?tag=a&tag=b`)
- → `useSearchParams()` returns a read-only object; copy to modify
- → `router.replace` avoids a history entry per filter change

WHERE EACH IS AVAILABLE

LOCATION	PARAMS	SEARCHPARAMS
`page.tsx`	✔ Promise	✔ Promise
`layout.tsx`	✔ Promise	✖
`route.ts`	✔ 2nd arg	`req.nextUrl.searchParams`
`generateMetadata`	✔ Promise	✔ Promise
`Client Component`	`useParams()`	`useSearchParams()`

TYPE HELPERS

HELPER	USE
`PageProps<'/a/[id]'>`	Page props
`LayoutProps<'/a'>`	Layout + slot props
`RouteContext<'/api/[id]'>`	Route handler 2nd arg
`Route`	Typed `href` (`typedRoutes`)

⚠ COMMON MISTAKES

- Assuming `searchParams.page` is a number; it is a string (or array) and must be parsed
- Reading `searchParams` in a layout to share filters; layouts don't receive it
- Using `useSearchParams()` without `<Suspense>`; the whole route falls back to client rendering

✓ TIPS & BEST PRACTICES

- Validate query values with Zod (`z.coerce.number()`) before using them
- Run `next typegen` in CI before `tsc` so route helpers exist
- Keep filter state in the URL so it's shareable and survives refresh

QUICK SUMMARY

- `await props.params` — dynamic segments
- `await props.searchParams` — query string (pages only)
- `PageProps<'/route'>` — generated types
- `useParams` / `useSearchParams` — client hooks

Static vs Dynamic Rendering

WHEN DOES A ROUTE RENDER?

A route is **static** when Next.js can render it at build time (or revalidate it in the background) and serve the result from a CDN. It becomes **dynamic** when it reads request-specific data: `cookies()`, `headers()`, `searchParams`, `connection()`, or uncached data. With **Cache Components**, a single route can be both: a static shell with dynamic holes.

DYNAMIC TRIGGERS

- `await cookies()` / `await headers()`
- `await searchParams` in a page
- `await connection()` (explicit opt-in)
- Uncached `fetch` or DB query (with Cache Components)

EXPLICITLY DEFER TO REQUEST TIME

```tsx
import { Suspense } from 'react'
import { connection } from 'next/server'
import { cookies } from 'next/headers'

export default function Page() {
  return (
    <>
      <Hero />                                  {/* static */}
      <Suspense fallback={<p>…</p>}><Greeting /></Suspense>
      <Suspense fallback={<p>…</p>}><RandomTip /></Suspense>
    </>
  )
}
async function Greeting() {
  const name = (await cookies()).get('name')?.value ?? 'friend'
  return <p>Welcome back, {name}</p>           // per request
}
async function RandomTip() {
  await connection()                           // don't prerender
  return <p>{tips[Math.floor(Math.random() * tips.length)]}</p>
}
```

NOTES

- → `<Hero />` is part of the prerendered shell
- → `cookies()` makes `Greeting` dynamic
- → `connection()` — use before `Math.random()` or `Date.now()`
- → Each dynamic part needs its own `<Suspense>` boundary
- → All request APIs are async in 16; sync access was removed

NEXT BUILD ROUTE SYMBOLS

SYMBOL	MEANING
○ (Static)	Prerendered as static content
● (SSG)	Prerendered with `generateStaticParams`
◐ (Partial Prerender)	Static shell + streamed dynamic parts
ƒ (Dynamic)	Rendered on demand per request

PICK A STRATEGY

CONTENT	STRATEGY
Marketing, docs	Static
Blog with edits	Static + revalidate / tags
Product page + live stock	Shell + Suspense hole
Dashboard per user	Dynamic (cookies)
Random / time-based	`connection()`

⚠ COMMON MISTAKES

- Reading `cookies()` in the root layout; every route becomes dynamic
- Calling `Date.now()` during prerender without `connection()`; the build captures one frozen value
- Assuming `'use client'` makes a route dynamic; Client Components still prerender

✓ TIPS & BEST PRACTICES

- Read request data as deep in the tree as possible so the shell stays static
- Check the symbols in `next build` output after every caching change
- Use `--debug-prerender` to see which component forced a route dynamic

QUICK SUMMARY

- Static — rendered at build, served from cache
- Dynamic — `cookies`, `headers`, `searchParams`
- `connection()` — explicit request-time render
- ◐ — partial prerender: shell + holes

Cache Components & 'use cache'

EXPLICIT, OPT-IN CACHING

With `cacheComponents: true`, nothing is cached unless you say so. Add the `'use cache'` directive to an async **function**, **component**, or a whole **file** to cache its return value. The cache key is built automatically from the build ID, the function's identity, and its **serialized arguments** (including captured closure variables).

CONSTRAINTS

- Cached functions must be `async`
- Arguments and return values must be serializable
- No `cookies()` / `headers()` / `searchParams` inside
- Entries don't survive a new deploy (key includes build ID)

THREE LEVELS OF CACHING

```ts
// next.config.ts → { cacheComponents: true }
import { cacheLife, cacheTag } from 'next/cache'

export async function getProduct(id: string) {     // function level
  'use cache'
  cacheLife('hours'); cacheTag(`product-${id}`)
  return db.product.findUnique({ where: { id } })
}

export async function Footer() {                   // component level
  'use cache'
  cacheLife('max')
  const links = await cms.getFooterLinks()
  return <footer>{/* … */}</footer>
}

// 'use cache' as first line of a file → every export is cached
```

NOTES

→ `id` is part of the cache key: one entry per product

→ `cacheLife` — how long the entry lives (sheet 23)

→ `cacheTag` — label for on-demand invalidation

→ Read cookies outside, pass values in as arguments

→ `children` / JSX props pass through without affecting the key

CACHE DIRECTIVES

DIRECTIVE	STORED	CAN READ COOKIES
`'use cache'`	In-memory / cacheHandlers	✖
`'use cache: remote'`	Remote handler (Redis, KV)	✖
`'use cache: private'`	Browser memory only	✔

WHICH ONE?

USE	SITUATION
`'use cache'`	Data for the static shell
`'use cache: remote'`	Shared, request-time, serverless
`'use cache: private'`	Per-user; can't refactor
`unstable_cache`	Must survive deploys
`NEXT_PRIVATE_DEBUG_CACHE=1`	Log cache hits/misses

⚠ COMMON MISTAKES

- Calling `cookies()` inside a cached function; it throws `next-request-in-use-cache`
- Passing a promise of request data into a cached component; the build hangs then times out after 50 s
- Relying on default in-memory `'use cache'` for runtime hits on serverless; instances don't share memory

✓ TIPS & BEST PRACTICES

- Always call `cacheLife()` in each cached scope so its lifetime is obvious
- Cache on low-cardinality keys (category, language), then filter in memory
- All three directives are enabled by `cacheComponents` (stable since 16.0)

QUICK SUMMARY

- `cacheComponents: true` — opt into the model
- `'use cache'` — cache a function, component, or file
- Key — build ID + function ID + arguments
- `remote` / `private` — shared store / browser only

cacheLife & cacheTag

cacheLife('hours') · built-in profiles · custom profiles · cacheTag() · stale · revalidate · expire

LIFETIMES AND LABELS

`cacheLife()` sets how long a `'use cache'` entry stays fresh, using a **profile** name or an inline object with three timings: `stale` (client may reuse without asking), `revalidate` (server refreshes in the background after this), and `expire` (after this with no traffic, the next request waits). `cacheTag()` labels entries so they can be invalidated on demand.

RULES

- Call both inside a cache scope, never at module top level
- No `cacheLife` → the `default` profile
- Client enforces a **30 s minimum** stale time
- `expire` must be longer than `revalidate`

PROFILES, INLINE, AND TAGS

```ts
// next.config.ts
const nextConfig: NextConfig = {
  cacheComponents: true,
  cacheLife: {
    editorial: { stale: 600, revalidate: 3600, expire: 86400 },
  },
}

// lib/posts.ts
import { cacheLife, cacheTag } from 'next/cache'
export async function getPost(slug: string) {
  'use cache'
  cacheTag('posts', `post-${slug}`)        // multiple tags allowed
  const post = await cms.post(slug)
  if (!post) { cacheLife('minutes'); return null }  // retry soon
  cacheLife('editorial')                   // custom profile
  return post
}
```

NOTES

- → Custom profiles inherit missing fields from `default`
- → Conditional `cacheLife` is fine; only one call may run
- → `cacheLife({ revalidate: 60 })` — inline one-off profile
- → Tag both the list (`posts`) and the item (`post-x`)
- → Profile names are type-checked after `next typegen`

BUILT-IN PROFILES

PROFILE	STALE	REVALIDATE	EXPIRE
default	5 min	15 min	never
seconds	30 s	1 s	1 min
minutes	5 min	1 min	1 h
hours	5 min	1 h	1 day
days	5 min	1 day	1 week
weeks	5 min	1 week	30 days
max	5 min	30 days	1 year

PRERENDER THRESHOLDS

LIFETIME	RESULT
revalidate 0 or expire < 5 min	Dynamic hole (request time)
stale < 30 s	Excluded from prerender
30 s ≤ stale < 5 min	Prerendered, not in App Shell
Nested short cache, no outer cacheLife	Build error

⚠ COMMON MISTAKES

- Nesting a `cacheLife('seconds')` component inside a cached parent without its own `cacheLife`; the build errors
- Wrapping `cacheLife` in a shared helper; keep it at the call site so behavior is readable
- Tagging with unbounded values like timestamps; tags should map to data you can name later

✓ TIPS & BEST PRACTICES

- Use `max` plus tags for content that only changes on edits
- Use `hours` or `days` for content that should refresh on its own
- Wrap short-lived cached components in `<Suspense>` since they become dynamic holes

QUICK SUMMARY

- `cacheLife(profile)` — stale / revalidate / expire
- Custom profiles — `cacheLife` key in config
- `cacheTag(...tags)` — label for invalidation
- 30 s — minimum client stale time

Revalidation

REFRESHING CACHED DATA

Cached data refreshes in two ways: **time-based** (from `cacheLife`) and **on-demand** after a mutation. Next.js 16 splits on-demand APIs by intent: `revalidateTag(tag, 'max')` marks entries stale and refreshes them in the background, `updateTag(tag)` expires them immediately so the user sees their own write, and `refresh()` re-renders uncached data only.

WHERE THEY WORK

- `updateTag` / `refresh` — Server Actions only
- `revalidateTag` / `revalidatePath` — Actions and Route Handlers
- Calling any of them in an Action clears the client router cache
- Single-argument `revalidateTag(tag)` is deprecated

CHOOSING THE RIGHT CALL

```ts
'use server'
import { updateTag, revalidateTag, revalidatePath, refresh } from 'next/cache'

export async function saveProfile(form: FormData) {
  const user = await requireUser()
  await db.user.update({ where: { id: user.id }, data: parse(form) })
  updateTag(`user-${user.id}`)       // read-your-writes, right now
}

export async function publishPost(id: string) {
  await db.post.update({ where: { id }, data: { published: true } })
  revalidateTag('posts', 'max')     // stale-while-revalidate for readers
}

export async function markRead(id: string) {
  await db.notification.update({ where: { id }, data: { read: true } })
  refresh()                         // re-render uncached badge count
}
```

NOTES

- → `updateTag` — next render waits for fresh data
- → `revalidateTag(tag, 'max')` — serve stale, refresh in background
- → Second arg: profile name or `{ expire: 3600 }`
- → `revalidatePath('/blog', 'layout')` — include nested routes

REVALIDATION APIS

API	SEMANTICS	WHERE
`updateTag(tag)`	Expire now; read-your-writes	Server Actions
`revalidateTag(tag, p)`	Stale-while-revalidate	Actions, handlers
`revalidatePath(path)`	Invalidate a route	Actions, handlers
`refresh()`	Re-render uncached data	Server Actions

CMS WEBHOOK PATTERN

STEP	DETAIL
Endpoint	`app/api/revalidate/route.ts`
Verify	Check a shared secret header
Invalidate	`revalidateTag(body.tag, 'max')`
Respond	`Response.json({ ok: true })`

⚠ COMMON MISTAKES

- Using `revalidateTag` after a form save and wondering why the user sees old data; use `updateTag`
- Calling `updateTag` in a Route Handler; it's only allowed in Server Actions
- Revalidating `/blog/[slug]` literally instead of the real path `/blog/hello` (or pass `'page'` type)

✓ TIPS & BEST PRACTICES

- Tag at the data layer, then invalidate by tag; it scales better than tracking paths
- Use `'max'` as the default second argument unless you need a custom window
- Protect revalidation webhooks with a secret and rate limiting

QUICK SUMMARY

- `updateTag` — immediate, Server Actions only
- `revalidateTag(tag, 'max')` — background refresh
- `revalidatePath` — invalidate by URL
- `refresh()` — uncached data only

Partial Prerendering

ONE ROUTE, STATIC AND DYNAMIC

Partial Prerendering (PPR) is how Cache Components renders a route: at build time Next.js prerenders a **static shell** containing all static and cached UI, plus the <Suspense> fallbacks for anything dynamic. At request time the shell is served immediately (often from a CDN) and the dynamic **holes** stream in. There's no separate PPR flag anymore.

16.X CHANGES

- `experimental.ppr` and `experimental_ppr` export removed
- Enable with `cacheComponents: true`
- 16.3 `partialPrefetching: true` prefetches each route's App Shell
- Dev overlay flags navigations that would block

SHELL + HOLES

```tsx
// app/product/[id]/page.tsx   (cacheComponents: true)
export default function Page(props: PageProps<'/product/[id]'>) {
  return (
    <>
      <SiteHeader />                                {/* static */}
      <Suspense fallback={<DetailsSkeleton />}>
        <Details params={props.params} />           {/* cached */}
      </Suspense>
      <Suspense fallback={<p>Checking stock…</p>}>
        <Stock params={props.params} />             {/* dynamic */}
      </Suspense>
    </>
  )
}
async function Details({ params }: { params: Promise<{ id: string }> }) {
  const { id } = await params
  return <ProductInfo p={await getProduct(id)} />    // 'use cache'
}
```

WHAT ENDS UP WHERE

- → Shell: header, skeletons, and cached `Details` for prerendered IDs
- → Hole: `Stock` queries live inventory per request
- → Uncached data outside `<Suspense>` → dev/build error
- → Unlisted IDs serve the shell first, then fill the cache (16.3)
- → Opt a segment out of validation: `export const instant = false`

WHAT GOES IN THE SHELL

CONTENT	IN SHELL?
Static JSX	✔
'use cache' (normal lifetime)	✔
Suspense fallbacks	✔
cookies() / headers() readers	✖ (hole)
Uncached fetch / DB	✖ (hole)
cacheLife('seconds')	✖ (hole)

16.3 INSTANT NAVIGATION TOOLS

TOOL	PURPOSE
Instant Insights	Dev panel listing slow navigations
Navigation Inspector	Pause on the loading shell
@next/playwright instant()	Assert shell content in tests
prefetch={true}	Per-link data prefetch

⚠ COMMON MISTAKES

- Placing one `<Suspense>` around the whole page; the shell becomes a single spinner
- Awaiting `params` at the top of the page; it blocks the shell for dynamic routes
- Expecting shell content from `'use cache: private'`; private caches are never in the static shell

✓ TIPS & BEST PRACTICES

- Pass `params` promises down and await them inside the boundary
- Push Suspense boundaries close to the data so more real content lands in the shell
- Use the Navigation Inspector to see exactly what users see before data arrives

QUICK SUMMARY

- Static shell — prerendered at build
- Dynamic holes — `<Suspense>` streamed per request
- Enabled by `cacheComponents: true`
- `partialPrefetching` — App Shell prefetch (16.3)

Previous Caching Model

WITHOUT CACHE COMPONENTS

Apps that don't enable `cacheComponents` still use the **previous model**: caching through `fetch` options, the **Data Cache**, and **route segment config** exports. It remains supported in 16.x. When you turn on Cache Components, `dynamic`, `revalidate`, `fetchCache`, and `dynamicParams` segment exports are no longer available, so plan the migration.

KEY DIFFERENCES

- Old: implicit static unless a dynamic API is used
- New: dynamic unless you add `'use cache'`
- Old: ISR via `revalidate` export
- New: ISR via `cacheLife` profiles

PREVIOUS-MODEL PATTERNS

```
// Data Cache with time-based and tag revalidation
const res = await fetch('https://api.example.com/posts', {
  next: { revalidate: 3600, tags: ['posts'] },
})

// Route segment config (page.tsx / layout.tsx / route.ts)
export const revalidate = 60              // ISR every 60 s
export const dynamic = 'force-static'     // or 'force-dynamic'
export const fetchCache = 'default-cache'

// Non-fetch data
import { unstable_cache } from 'next/cache'
const getStats = unstable_cache(
  async () => db.stats.aggregate(),
  ['stats'], { revalidate: 300, tags: ['stats'] })
```

NOTES

- → `next.revalidate` — seconds before background refetch
- → `tags` — invalidate with `revalidateTag('posts', 'max')`
- → `force-static` — treat dynamic APIs as empty at build
- → `unstable_cache` persists across deploys (unlike `'use cache'`)

MIGRATION MAP

PREVIOUS	CACHE COMPONENTS
`export const revalidate = 60`	`cacheLife({ revalidate: 60 })`
`dynamic = 'force-static'`	`'use cache'` on the page
`dynamic = 'force-dynamic'`	Default (remove it)
`fetch(..., { cache: 'force-cache' })`	`'use cache'` around the fetch
`unstable_cache(fn, keys, opts)`	`'use cache'` + `cacheLife` + `cacheTag`
`experimental_ppr`	Removed; Suspense + cacheComponents

SEGMENT CONFIG STILL VALID

EXPORT	VALUES
`runtime`	`'nodejs'` (default), `'edge'` deprecated
`maxDuration`	Seconds (platform limit)
`dynamicParams`	Previous model only
`instant`	16.3; `false` opts out of validation

⚠ COMMON MISTAKES

- Enabling `cacheComponents` while pages still export `revalidate` or `dynamic`; the build reports them
- Assuming `fetch` is cached by default as in Next 14; it has been uncached since 15
- Migrating everything at once; adopt route by route following the dev overlay

✓ TIPS & BEST PRACTICES

- Use the official guide 'Migrating to Cache Components' and its agent Skill for large apps
- Keep `unstable_cache` for data that must persist across deploys
- Search the codebase for `export const dynamic` before flipping the flag

QUICK SUMMARY

- `next: { revalidate, tags }` — Data Cache
- `export const revalidate` — route ISR
- `unstable_cache` — non-fetch caching
- Cache Components removes segment caching exports

Server Actions Basics

FUNCTIONS THAT RUN ON THE SERVER

A **Server Action** (Server Function) is an async function marked with `'use server'`. Pass it to `<form action>` or call it from a Client Component, and Next.js sends a POST request, runs it on the server, and returns the result. Actions are the App Router's way to handle **mutations** without writing API routes.

BASICS

- Forms work before JS loads (progressive enhancement)
- Arguments and return values must be serializable
- Run one at a time per client (sequential queue)
- Usually end with `updateTag`, `revalidatePath`, or `redirect`

ACTION FILE + FORM

```tsx
// app/todos/actions.ts
'use server'                          // every export is an action
import { updateTag } from 'next/cache'
import { redirect } from 'next/navigation'

export async function createTodo(formData: FormData) {
  const user = await requireUser()    // always authorize
  const title = String(formData.get('title') ?? '').trim()
  const todo = await db.todo.create({ data: { title, userId: user.id } })
  updateTag(`todos-${user.id}`)
  redirect(`/todos/${todo.id}`)       // throws; call last
}

// app/todos/new/page.tsx  (Server Component)
import { createTodo } from '../actions'
export default function NewTodo() {
  return <form action={createTodo}><input name="title" /></form>
}
```

NOTES

- → File-level `'use server'` — every export becomes an action
- → `formData.get()` — read fields by `name` attribute
- → `redirect()` in an action responds with 303
- → Call `redirect` outside `try/catch`, after the write

INLINE ACTION + BOUND ARGS

```tsx
export default async function Post({ id }: Props) {
  async function like() {
    'use server'            // inline action
    await db.like.create({ data: { postId: id } })
  }
  const remove = deletePost.bind(null, id) // bind
  return <>
    <form action={like}><button>♥</button></form>
    <form action={remove}><button>Delete</button></form>
  </>
}
```

WAYS TO INVOKE

FROM	HOW
`<form action>`	Receives `FormData`
`<button formAction>`	Per-button action
`Client onClick`	`startTransition(() => act(x))`
`useActionState`	Adds prev state + pending

⚠ COMMON MISTAKES

- Skipping authorization because the form is on a protected page; actions are public POST endpoints
- Calling `redirect()` inside `try`; the thrown redirect is caught and never happens
- Defining `'use server'` functions inside Client Components; define them in a separate file and import

✓ TIPS & BEST PRACTICES

- Keep actions in `actions.ts` files per feature so they're easy to audit
- Use `.bind(null, id)` to pass IDs; bound args are encrypted, unlike hidden inputs
- Return plain objects (`{ ok, error }`) for expected errors instead of throwing

QUICK SUMMARY

- `'use server'` — marks Server Actions
- `<form action={fn}>` — progressive enhancement
- `FormData` — input values by `name`
- Finish with `updateTag` or `redirect`

Forms & Pending States

FEEDBACK WHILE THE SERVER WORKS

React 19 hooks turn Server Actions into full form workflows. `useActionState` stores the action's last return value (errors, messages) and a **pending** flag. `useFormStatus` reads the pending state of the parent form from a child button. `useOptimistic` shows the expected result before the server responds. Validate input on the server with **Zod**.

VALIDATION PATTERN

- Parse `FormData` with `safeParse`
- Return `{ errors }` instead of throwing
- Re-fill inputs with `defaultValue` from state
- Client `required` is UX only, not security

ACTION WITH ZOD

```
'use server'
import { z } from 'zod'
const Schema = z.object({
  email: z.email(),
  name: z.string().min(2, 'Too short'),
})
export type State = {
  errors?: Record<string, string[]>; ok?: boolean }

export async function signup(
  _prev: State, form: FormData): Promise<State> {
  const r = Schema.safeParse(Object.fromEntries(form))
  if (!r.success)
    return { errors: z.flattenError(r.error).fieldErrors }
  await db.user.create({ data: r.data })
  return { ok: true }
}
```

CLIENT FORM

```
'use client'
import { useActionState } from 'react'
import { useFormStatus } from 'react-dom'
import { signup, type State } from './actions'

function Submit() {
  const { pending } = useFormStatus() // parent form
  return <button disabled={pending}>Sign up</button>
}
export function SignupForm() {
  const init: State = {}
  const [state, action] = useActionState(signup, init)
  const e = state.errors
  return <form action={action}>
    <input name="email" /><p>{e?.email?.[0]}</p>
    <input name="name" /><p>{e?.name?.[0]}</p>
    <Submit />{state.ok && <p>Welcome!</p>}
  </form>
}
```

FORM HOOKS

HOOK	FROM	RETURNS
`useActionState(fn, init)`	react	[state, action, pending]
`useFormStatus()`	react-dom	{ pending, data, method }
`useOptimistic(state, fn)`	react	[optimistic, add]
`useTransition()`	react	[pending, start]

NEXT/FORM <FORM>

FEATURE	DETAIL
`action="/search"`	GET: navigates with query params
`Prefetch`	Prefetches the target's loading UI
`action={fn}`	Behaves like a normal React form
`Use for`	Search boxes, filters

⚠ COMMON MISTAKES

- Calling `useFormStatus` in the same component that renders the `<form>`; it only reads a parent form
- Forgetting that the action's first parameter becomes previous state when used with `useActionState`
- Trusting client validation alone; server parsing is the only real check

✓ TIPS & BEST PRACTICES

- Share one Zod schema between client hints and server validation
- Use `<Form action="/search">` from `next/form` for GET search forms with client navigation
- Return field values in state so inputs keep user input after an error

QUICK SUMMARY

- `useActionState` — return value + pending
- `useFormStatus` — pending from a child component
- `useOptimistic` — instant UI before confirmation
- Zod `safeParse` — return errors, don't throw

Server Action Security

TREAT EVERY ACTION LIKE AN API ROUTE

Each Server Action is reachable by a **public POST request** with a hard-to-guess ID. Hiding a button or protecting a page does **not** protect the action. Inside every action: **authenticate**, **authorize** the specific resource, **validate** input, and return only what the client needs.

BUILT-IN PROTECTIONS

- Origin check: `Origin` must match `Host`
- Unused actions are removed from the bundle
- Action IDs are non-deterministic per build
- Closed-over values are encrypted

A HARDENED ACTION

```
'use server'
import 'server-only'
import { z } from 'zod'
import { verifySession } from '@/lib/dal'
import { forbidden } from 'next/navigation'

const Input = z.object({ projectId: z.uuid(), name: z.string().max(80) })

export async function renameProject(input: unknown) {
  const session = await verifySession()                // 1. authenticate
  const { projectId, name } = Input.parse(input)       // 2. validate
  const project = await db.project.findUnique({ where: { id: projectId } })
  if (project?.ownerId !== session.userId) forbidden() // 3. authorize
  await db.project.update({ where: { id: projectId }, data: { name } })
  updateTag(`project-${projectId}`)
  return { ok: true }                                  // 4. minimal result
}
```

CHECKLIST

- → `input: unknown` — never trust the declared TS type
- → Check ownership of the specific row, not just login
- → `forbidden()` needs `experimental.authInterrupts`
- → Return IDs and flags, not whole DB records
- → Rate-limit expensive or sensitive actions

SERVERACTIONS CONFIG

OPTION	PURPOSE
`bodySizeLimit`	Max request body (default `1mb`)
`allowedOrigins`	Extra origins behind proxies
`NEXT_SERVER_ACTIONS_ENCRYPTION_KEY`	Shared key across instances
`Location`	`experimental.serverActions` in config

COMMON RISKS

RISK	MITIGATION
`Missing auth check`	Call `verifySession()` first
`IDOR (other users' data)`	Check ownership per record
`Mass assignment`	Whitelist fields with Zod
`Leaking data in return`	Return DTOs only
`Proxy matcher skips path`	Never rely on proxy alone

⚠ COMMON MISTAKES

- Relying on `proxy.ts` for action auth; actions POST to the page URL and matcher changes can skip them
- Accepting a `userId` argument from the client instead of reading it from the session
- Spreading `formData` straight into an ORM `update` call; attackers can set any column

✓ TIPS & BEST PRACTICES

- Set `NEXT_SERVER_ACTIONS_ENCRYPTION_KEY` when running multiple instances so encrypted closures decrypt everywhere
- Put all data access in a Data Access Layer that checks auth itself
- Log action failures with the user ID for audit trails

QUICK SUMMARY

- Actions are public endpoints
- Authenticate, authorize, validate — every time
- `bodySizeLimit` / `allowedOrigins` — tune limits
- Return DTOs, never raw records

Route Handlers

HTTP ENDPOINTS IN APP/

A `route.ts` file exports functions named after **HTTP methods**. Each receives a Web `Request` (or `NextRequest`) and returns a `Response`. Route Handlers replace Pages Router API routes and are the right tool for webhooks, public APIs, file downloads, and endpoints that non-React clients call.

RULES

- Supported: GET, POST, PUT, PATCH, DELETE, HEAD, OPTIONS
- Unsupported methods return 405 automatically
- Can't live in the same folder as `page.tsx`
- GET handlers are dynamic by default

CRUD ENDPOINT

```ts
// app/api/posts/[id]/route.ts
import { NextRequest, NextResponse } from 'next/server'
type Ctx = RouteContext<'/api/posts/[id]'>

export async function GET(_req: NextRequest, ctx: Ctx) {
  const { id } = await ctx.params                       // Promise in 16
  const post = await db.post.findUnique({ where: { id } })
  if (!post) return NextResponse.json({ error: 'Not found' }, { status: 404 })
  return Response.json(post)                            // 200
}

export async function PUT(req: NextRequest, ctx: Ctx) {
  const { id } = await ctx.params
  const body = await req.json()
  const post = await db.post.update({ where: { id }, data: body })
  return Response.json(post)
}

export async function DELETE(_req: Request, ctx: Ctx) {
  await db.post.delete({ where: { id: (await ctx.params).id } })
  return new Response(null, { status: 204 })
}
```

NOTES

- → `RouteContext<'/api/posts/[id]'>` — typed `params` helper
- → `Response.json()` — standard Web API; works everywhere
- → `NextResponse.json()` — same plus cookie helpers
- → `204` responses must have a `null` body
- → Validate `body` before writing (Zod)

NEXTREQUEST EXTRAS

PROPERTY	RETURNS
`req.nextUrl.pathname`	`/api/posts/1`
`req.nextUrl.searchParams`	`URLSearchParams`
`req.cookies.get('id')`	`{ name, value }`
`req.headers.get('x')`	Header value
`await req.json()`	Parsed JSON body
`await req.formData()`	`FormData`

BUILDING RESPONSES

CALL	RESULT
`Response.json(data)`	200 JSON
`Response.json(d, { status: 201 })`	Created
`NextResponse.redirect(url)`	307 redirect
`new Response(stream)`	Streamed body
`res.cookies.set(n, v)`	Set-Cookie (NextResponse)

⚠ COMMON MISTAKES

- Calling your own Route Handler from a Server Component; call the data function directly
- Expecting `GET` responses to be cached by default; they're dynamic unless you opt in
- Using a Route Handler for your own form submissions where a Server Action is simpler

✓ TIPS & BEST PRACTICES

- Use `RouteContext` for typed params instead of hand-written types
- Return consistent error shapes (`{ error }`) with correct status codes
- Put handlers under `app/api/` by convention so they're easy to exclude in `proxy.ts` matchers

QUICK SUMMARY

- `route.ts` — export GET, POST, etc.
- `await ctx.params` — dynamic segments
- `Response.json()` — JSON responses
- `NextRequest` — `nextUrl`, `cookies` helpers

Route Handler Patterns

REAL-WORLD ENDPOINTS

Route Handlers use standard Web APIs, so most patterns are plain `Request`/`Response` code. Next.js adds helpers for **cookies** and **headers**, supports **streaming** responses (useful for AI output and large exports), and, with Cache Components, lets you cache GET logic with `'use cache'` in a helper function.

PATTERNS

- Webhooks: verify the signature on the **raw** body
- CORS: handle `OPTIONS` preflight explicitly
- Streaming: return a `ReadableStream`
- Cache GET work with a `'use cache'` helper

VERIFIED WEBHOOK

```ts
// app/api/stripe/route.ts
import Stripe from 'stripe'
import { revalidateTag } from 'next/cache'
const stripe = new Stripe(process.env.STRIPE_KEY!)

export async function POST(req: Request) {
  const raw = await req.text()            // raw body
  const sig = req.headers.get('stripe-signature')!
  let event: Stripe.Event
  try {
    event = stripe.webhooks.constructEvent(
      raw, sig, process.env.STRIPE_WEBHOOK_SECRET!)
  } catch {
    return new Response('Bad signature', { status: 400 })
  }
  if (event.type === 'product.updated')
    revalidateTag('products', 'max')
  return Response.json({ received: true })
}
```

STREAMING + CORS

```ts
// app/api/stream/route.ts
export async function GET() {
  const enc = new TextEncoder()
  const stream = new ReadableStream({
    async start(ctrl) {
      for (const word of ['Hello', ' ', 'world']) {
        ctrl.enqueue(enc.encode(word))
        await new Promise((r) => setTimeout(r, 200))
      }
      ctrl.close()
    },
  })
  return new Response(stream, {
    headers: { 'Content-Type': 'text/plain' } })
}

export function OPTIONS() {        // CORS preflight
  return new Response(null, { status: 204, headers: {
    'Access-Control-Allow-Origin': 'https://app.acme.com',
    'Access-Control-Allow-Methods': 'GET, POST' } })
}
```

COOKIES & HEADERS

CALL	EFFECT
`(await cookies()).get('t')`	Read a cookie
`(await cookies()).set('t', v)`	Set cookie on response
`(await headers()).get('ua')`	Read request header
`new Response(b, { headers })`	Set response headers

READING BODIES

BODY TYPE	METHOD
`JSON`	`await req.json()`
`Form / multipart`	`await req.formData()`
`Raw text (webhooks)`	`await req.text()`
`Binary`	`await req.arrayBuffer()`

⚠ COMMON MISTAKES

- Calling `req.json()` before verifying a webhook signature; the signature covers the raw text
- Reading the body twice; a request body stream can only be consumed once
- Using `Access-Control-Allow-Origin: *` with credentials; browsers reject it

✓ TIPS & BEST PRACTICES

- Return `new Response(stream)` for long exports so memory stays flat
- Use `after()` to log or send analytics without delaying the response
- Set `export const maxDuration = 60` for slow handlers on platforms that support it

QUICK SUMMARY

- `req.text()` — raw body for signatures
- `ReadableStream` — streamed responses
- `OPTIONS` — CORS preflight
- `cookies()` / `headers()` — async helpers

proxy.ts (formerly middleware.ts)

CODE BEFORE EVERY REQUEST

`proxy.ts` (at the project root or in `src/`) runs **before routing** for every matching request. It can redirect, rewrite, set headers and cookies, or answer directly. Next.js 16 renamed `middleware.ts` to `proxy.ts` to make the network boundary clear, and Proxy runs on the **Node.js runtime**.

16.X FACTS

- Export `proxy` (named) or a default function
- `runtime` config isn't allowed in proxy files
- `middleware.ts` still works but is deprecated (Edge only)
- Codemod: `npx @next/codemod@canary middleware-to-proxy .`

AUTH REDIRECT + LOCALE REWRITE

```ts
// proxy.ts
import { NextResponse, type NextRequest } from 'next/server'
export function proxy(request: NextRequest) {
  const { pathname } = request.nextUrl
  const loggedIn = request.cookies.has('session')     // optimistic check
  if (pathname.startsWith('/dashboard') && !loggedIn) {
    return NextResponse.redirect(new URL('/login', request.url))
  }
  if (pathname === '/') {
    const al = request.headers.get('accept-language') ?? ''
    const lang = al.startsWith('de') ? 'de' : 'en'
    return NextResponse.rewrite(new URL(`/${lang}`, request.url))
  }
  const res = NextResponse.next(); res.headers.set('x-pathname', pathname)
  return res
}

export const config = {
  matcher: ['/((?!api|_next/static|_next/image|favicon.ico).*)'],
}
```

NOTES

- → `redirect` — changes the browser URL
- → `rewrite` — serves `/en` while the URL stays `/`
- → `next()` — continue, optionally adding headers
- → Cookie check is a fast hint; verify sessions again in code

MATCHER SYNTAX

PATTERN	MATCHES
`'/about'`	`/about` and below
`'/blog/:slug'`	One segment
`'/docs/:path*'`	Zero or more segments
`'/((?!api).*)'`	All but `/api` (regex)
`{ source, has, missing }`	Conditional on header/cookie

EXECUTION ORDER

#	STEP
1	`headers` from next.config
2	`redirects` from next.config
3	**proxy.ts**
4	`beforeFiles` rewrites
5	Filesystem, then dynamic routes

⚠ COMMON MISTAKES

- Doing database lookups in proxy for every request; it adds latency to every page and asset
- Relying on proxy as the only auth layer; Server Actions and handlers must check again
- Building matchers from variables; matchers must be static constants

✓ TIPS & BEST PRACTICES

- Exclude `_next/static`, `_next/image`, and metadata files in the matcher
- Pass data downstream with request headers via `NextResponse.next({ request: { headers } })`
- Unit test matchers with `unstable_doesProxyMatch` from `next/experimental/testing/server`

QUICK SUMMARY

- `proxy.ts` — runs before routing (Node.js)
- `config.matcher` — limit where it runs
- `redirect` / `rewrite` / `next` — responses
- Renamed from `middleware.ts` in 16

Request-Time APIs

cookies() · headers() · draftMode() · connection() · after() · all async

READING THE INCOMING REQUEST

Server Components, Server Actions, and Route Handlers read request data through functions from `next/headers` and `next/server`. Since Next.js 16 they are **async only**: always `await` them. Calling any of them makes that part of the tree render at request time, so wrap it in `<Suspense>` when using Cache Components.

WHERE WRITES ARE ALLOWED

- `cookies().set/delete` — Server Actions, Route Handlers
- Not in Server Components (headers already sent)
- `draftMode().enable()` — Route Handlers or Actions
- `after()` — anywhere on the server

USING EACH API

```ts
import { cookies, headers, draftMode } from 'next/headers'
import { connection, after } from 'next/server'

const store = await cookies()
store.get('theme')?.value                    // 'dark'
store.has('session')                         // true
store.set('theme', 'light', { httpOnly: true, maxAge: 60 * 60 * 24 })
store.delete('session')                      // Actions / handlers only

const h = await headers()
h.get('user-agent')                          // 'Mozilla/5.0 …'
h.get('x-forwarded-for')                     // client IP behind proxy

const { isEnabled } = await draftMode()      // CMS preview on?
await connection()                           // force request-time render

after(async () => {                          // runs after response is sent
  await analytics.track('page_view', { path: '/pricing' })
})
```

NOTES

- → Every call returns a Promise in 16 (sync access removed)
- → `headers()` is read-only; set response headers elsewhere
- → `draftMode` bypasses `'use cache'` while enabled
- → `after()` — logging without slowing the user down
- → None of these can run inside `'use cache'`

COOKIE OPTIONS

OPTION	PURPOSE
`httpOnly`	Hide from JavaScript
`secure`	HTTPS only
`sameSite`	`'lax'` / `'strict'` / `'none'`
`maxAge / expires`	Lifetime
`path / domain`	Scope

AVAILABILITY

API	RSC	ACTION	HANDLER
`cookies() read`	✔	✔	✔
`cookies() write`	✖	✔	✔
`headers()`	✔	✔	✔
`draftMode().enable()`	✖	✔	✔
`after()`	✔	✔	✔

⚠ COMMON MISTAKES

- Setting cookies from a Server Component; it throws because streaming has already started
- Forgetting `await` (`cookies().get(...)`); in 16 that's a type and runtime error
- Reading `headers()` in a shared layout and making every page dynamic

✓ TIPS & BEST PRACTICES

- Read cookies outside `'use cache'` functions and pass plain values in
- Use `after()` for audit logs and cache warming after a mutation
- Run `npx @next/codemod@canary next-async-request-api .` on older code

QUICK SUMMARY

- `await cookies()` — read/write cookies
- `await headers()` — read request headers
- `await connection()` — opt into request time
- `after(fn)` — work after the response

CSS, CSS Modules & Sass

STYLING OPTIONS BUILT IN

Next.js supports **global CSS**, **CSS Modules** (locally scoped class names), and **Sass** with no extra config beyond installing `sass`. CSS Modules are the safest default for component styles: every class is renamed to a unique hash, so names never collide. Global CSS is for resets, variables, and base element styles.

RULES

- Global CSS can be imported anywhere, but keep it in the root layout
- CSS Modules work in Server and Client Components
- Production builds bundle and minify CSS automatically
- Sass needs `npm i -D sass` (modern API in 16)

CSS MODULE

```
/* app/components/button.module.css */
.button { padding: 0.5rem 1rem; border-radius: 6px; }
.primary { background: var(--brand); color: white; }
.button:disabled { opacity: 0.5; }

// app/components/button.tsx
import styles from './button.module.css'
import clsx from 'clsx'

export function Button({ primary, ...props }: Props) {
  const c = clsx(styles.button, primary && styles.primary)
  return <button {...props} className={c} />
}
```

GLOBAL CSS + SASS

```
/* app/globals.css */
:root { --brand: #5c6bc0; }
*, *::before, *::after { box-sizing: border-box; }
body { font-family: var(--font-sans), sans-serif; }
/* import once in app/layout.tsx: './globals.css' */

/* app/card.module.scss */
$radius: 8px;
.card {
  border-radius: $radius;
  &:hover { box-shadow: 0 2px 8px rgb(0 0 0 / 0.2); }
}
```

WHICH APPROACH?

APPROACH	SCOPE	BEST FOR
`globals.css`	Global	Resets, tokens, base styles
`*.module.css`	Component	Most component styles
`*.module.scss`	Component	Nesting, mixins, variables
`Tailwind`	Utility	Fast UI building (sheet 35)
`Inline style={{}}`	Element	Truly dynamic values

CSS ORDERING

FACT	DETAIL
`Import order`	Determines cascade order
`Dev vs prod`	Order can differ; test `next build`
`Layout CSS`	Loaded before page CSS
`Specificity`	Prefer single-class selectors

⚠ COMMON MISTAKES

- Relying on import order between two modules to win a cascade fight; it can differ in production
- Using `styles['my-class']` typos; a missing key is `undefined` and silently renders no class
- Importing a third-party global stylesheet inside a deep component instead of the root layout

✓ TIPS & BEST PRACTICES

- Use `clsx` (or `cn` with `tailwind-merge`) to combine conditional class names
- Define design tokens as CSS variables in `globals.css` and reference them in modules
- Use `:global(.class)` inside a module only for third-party overrides

QUICK SUMMARY

- `globals.css` — import once in root layout
- `*.module.css` — scoped class names
- `sass` — install, then use `.scss` files
- `clsx` — conditional class names

Tailwind CSS

UTILITY-FIRST STYLING

`create-next-app` sets up **Tailwind CSS v4** by default. Version 4 is configured in CSS rather than JavaScript: one `@import "tailwindcss"` line loads everything, and design tokens live in an `@theme` block. Classes are generated on demand from your source files, so only what you use ships.

V4 VS V3

- No `tailwind.config.js` required
- PostCSS plugin is `@tailwindcss/postcss`
- No `@tailwind base/components/utilities` lines
- Source files are detected automatically

SETUP FILES

```
/* postcss.config.mjs:
   plugins: { '@tailwindcss/postcss': {} } */

/* app/globals.css */
@import "tailwindcss";

@theme {
  --color-brand: #5c6bc0;
  --font-sans: var(--font-inter);   /* from next/font */
  --breakpoint-3xl: 120rem;
}

@custom-variant dark (&:where(.dark, .dark *));
```

USING IT

```
// bg-brand, font-sans, 3xl: now exist
export function Card({ title, featured }: Props) {
  return (
    <div className={cn(
      'rounded-lg p-4 shadow bg-white dark:bg-zinc-900',
      'md:p-6 3xl:p-10',
      featured && 'ring-2 ring-brand')}>
      <h2 className="font-sans text-lg font-semibold">
        {title}</h2>
    </div>
  )
}

// lib/utils.ts
import { clsx, type ClassValue } from 'clsx'
import { twMerge } from 'tailwind-merge'
export const cn = (...c: ClassValue[]) => twMerge(clsx(c))
```

EVERYDAY UTILITIES

CLASS	CSS
`flex items-center gap-2`	Flex row, centered, 0.5rem gap
`grid grid-cols-3`	3-column grid
`md:grid-cols-2`	Applies at ≥ 768px
`hover:bg-brand/90`	Hover with 90% opacity
`dark:text-white`	Dark mode variant
`size-8`	width + height 2rem

INTEGRATION NOTES

TOPIC	DETAIL
Dark mode	`next-themes` + `.dark` class variant
Fonts	`next/font` `variable` → `@theme`
Conflicts	`twMerge` resolves `p-2` vs `p-4`
Dynamic names	Write full class strings
Plugins	`@plugin "@tailwindcss/typography";`

⚠ COMMON MISTAKES

- Building class names with template strings (`bg-${color}-500`); the scanner can't see them, so they're never generated
- Keeping v3 `@tailwind` directives after upgrading; v4 uses a single `@import`
- Merging conflicting utilities with plain `clsx`; the last class in the CSS file wins, not the last in the string

✓ TIPS & BEST PRACTICES

- Map props to complete class strings (`{ red: 'bg-red-500' }`) for dynamic styling
- Use the `cn()` helper so component callers can override classes safely
- Install the Tailwind IntelliSense editor extension for autocomplete

QUICK SUMMARY

- `@import "tailwindcss"` — v4 setup
- `@theme` — tokens become utilities
- `@tailwindcss/postcss` — PostCSS plugin
- `cn()` = `clsx` + `twMerge`

CSS-in-JS & Component Libraries

LIBRARIES AND THE SERVER BOUNDARY

Runtime **CSS-in-JS** libraries (styled-components, Emotion) need React context and run in the browser, so they only work in **Client Components** and need a **style registry** to collect styles during server rendering. Zero-runtime options (CSS Modules, Tailwind, vanilla-extract, Panda) work in Server Components with no extra setup.

COMPATIBILITY

- styled-components: client only + registry
- Emotion: client only; limited App Router support
- vanilla-extract / Panda / StyleX: build-time, server-safe
- shadcn/ui: copied source using Tailwind + Radix

STYLED-COMPONENTS REGISTRY

```
// next.config.ts
const nextConfig: NextConfig = { compiler: { styledComponents: true } }

// lib/registry.tsx
'use client'
import { useState } from 'react'
import { useServerInsertedHTML } from 'next/navigation'
import { ServerStyleSheet, StyleSheetManager } from 'styled-components'

export function StyledRegistry({ children }: { children: React.ReactNode }) {
  const [sheet] = useState(() => new ServerStyleSheet())
  useServerInsertedHTML(() => {
    const styles = sheet.getStyleElement()
    sheet.instance.clearTag()
    return <>{styles}</>
  })
  if (typeof window !== 'undefined') return <>{children}</>
  return (
    <StyleSheetManager sheet={sheet.instance}>{children}</StyleSheetManager>)
}
// app/layout.tsx → <body><StyledRegistry>{children}</StyledRegistry></body>
```

HOW IT WORKS

- → `compiler.styledComponents` — SWC transform for SSR + display names
- → `useServerInsertedHTML` — inject collected `<style>` tags while streaming
- → `clearTag()` — avoid duplicating styles across chunks
- → Every styled component file still needs `'use client'`

SHADCN/UI WORKFLOW

COMMAND	RESULT
`npx shadcn@latest init`	Adds `components.json`, `cn` util
`npx shadcn@latest add button`	Copies `components/ui/button.tsx`
Edit freely	You own the source
Built on	Radix primitives + Tailwind

LIBRARY PICKS

LIBRARY	SERVER-SAFE STYLING
Radix / Base UI	Unstyled; style with anything
shadcn/ui	✔ (Tailwind)
Mantine	CSS Modules based; client providers
MUI	Use `@mui/material-nextjs` integration
Chakra UI	Client provider required

⚠ COMMON MISTAKES

- Using `styled.div` in a Server Component; runtime CSS-in-JS needs `'use client'`
- Skipping the registry; styles load after hydration and the page flashes unstyled
- Wrapping the whole app in a UI library provider when only a few widgets need it

✓ TIPS & BEST PRACTICES

- Prefer zero-runtime styling for new App Router projects
- Interactive primitives (dialogs, menus) are client components; keep the pages around them server-rendered
- Check each library's App Router docs for a ready-made Next.js integration package

QUICK SUMMARY

- Runtime CSS-in-JS — client only + registry
- `useServerInsertedHTML` — inject SSR styles
- Zero-runtime libraries — server-safe
- shadcn/ui — copy-in Radix + Tailwind components

next/image

OPTIMIZED IMAGES

`<Image>` from `next/image` resizes, compresses, and converts images to modern formats (WebP/AVIF) on demand, serves a responsive `srcset`, lazy-loads by default, and reserves space to prevent **layout shift**. Remote images must be allow-listed in `next.config`.

16.X CHANGES

- `priority` deprecated → use `preload`
- `images.qualities` defaults to `[75]`
- `minimumCacheTTL` default is now 4 hours
- Local `src` with query strings needs `localPatterns`

STATIC, REMOTE, AND FILL IMAGES

```tsx
import Image from 'next/image'
import hero from '@/public/hero.jpg'          // static import

<Image src={hero} alt="Mountain lake" placeholder="blur" preload />
// width/height/blurDataURL inferred from the file

<Image src="https://cdn.example.com/u/42.png" alt="Avatar"
  width={64} height={64} className="rounded-full" />

<div style={{ position: 'relative', aspectRatio: '16 / 9' }}>
  <Image src={post.cover} alt={post.title} fill
    sizes="(max-width: 768px) 100vw, 50vw"
    style={{ objectFit: 'cover' }} />
</div>

// next.config.ts
images: { remotePatterns: [new URL('https://cdn.example.com/**')],
          qualities: [50, 75, 90] }
```

NOTES

- → Static import — dimensions and blur placeholder are automatic
- → Remote `src` — `width` and `height` required (or `fill`)
- → `fill` — parent must be `position: relative` (or absolute/fixed)
- → `sizes` — tells the browser which `srcset` entry to pick
- → `preload` — only for the LCP image above the fold

KEY PROPS

PROP	PURPOSE
`alt`	Required; `""` for decorative
`width / height`	Intrinsic size in px
`fill`	Fill the parent box
`sizes`	Responsive width hints
`quality`	Snapped to `images.qualities`
`placeholder`	`"blur"` · `"empty"` · data URL
`preload`	Add `<link rel=preload>`
`unoptimized`	Serve the original file

IMAGES CONFIG

KEY	DEFAULT / USE
`remotePatterns`	Allowed remote hosts
`localPatterns`	Allowed local paths + queries
`qualities`	`[75]`
`formats`	`['image/webp']` (+ avif)
`minimumCacheTTL`	`14400` s (4 h)
`imageSizes / deviceSizes`	srcset widths
`loader / loaderFile`	Use an external CDN

⚠ COMMON MISTAKES

- Using `fill` without `sizes`; the browser downloads a full-viewport image for a small card
- Adding `preload` to many images; it competes for bandwidth with the real LCP image
- Expecting `quality={80}` to work without listing 80 in `images.qualities`; it snaps to 75

✓ TIPS & BEST PRACTICES

- Static-import local images to get dimensions and blur placeholders for free
- Use a `loaderFile` for Cloudinary/Imgix so their CDN does the resizing
- Self-hosting: `sharp` is used automatically for fast optimization

QUICK SUMMARY

- `<Image>` — responsive, lazy, optimized
- `fill` + `sizes` — fluid layouts
- `preload` — replaces `priority` for LCP
- `remotePatterns` — allow remote hosts

next/font

ZERO LAYOUT SHIFT FONTS

`next/font` downloads fonts at **build time** and serves them from your own domain: no requests to Google at runtime, better privacy, and automatic `size-adjust` fallbacks so text doesn't jump when the font loads. Call the loader at **module scope** and apply the returned `className` or CSS `variable`.

FACTS

- Variable fonts need no `weight` option
- Static fonts need explicit `weight` (s)
- `subsets` limits the preloaded character set
- Each loader call creates one font instance; share it

GOOGLE + LOCAL FONTS

```tsx
// app/fonts.ts
import { Inter, Roboto_Mono } from 'next/font/google'
import localFont from 'next/font/local'

export const inter = Inter({ subsets: ['latin'], variable: '--font-inter' })
export const mono = Roboto_Mono({ subsets: ['latin'], weight: ['400', '700'],
  variable: '--font-mono', display: 'swap' })
export const brand = localFont({
  src: [{ path: './fonts/Brand-Regular.woff2', weight: '400' },
        { path: './fonts/Brand-Bold.woff2', weight: '700' }],
  variable: '--font-brand' })

// app/layout.tsx
import { inter, mono, brand } from './fonts'
const vars = `${inter.variable} ${mono.variable} ${brand.variable}`
<html lang="en" className={vars}>
  <body className={inter.className}>{children}</body></html>
```

NOTES

- → `variable` — exposes a CSS custom property
- → `className` — applies `font-family` directly
- → Underscores replace spaces: `Roboto_Mono`
- → `localFont` paths are relative to the calling file
- → Use `var(--font-inter)` in CSS or Tailwind `@theme`

LOADER OPTIONS

OPTION	EXAMPLE
subsets	['latin', 'latin-ext']
weight	'400' or ['400', '700']
style	['normal', 'italic']
display	'swap' (default)
variable	'--font-inter'
preload	false for secondary fonts
adjustFontFallback	true (reduces CLS)

APPLYING FONTS

APPROACH	CODE
Whole app	<body className={inter.className}>
One element	<h1 className={brand.className}>
CSS var	font-family: var(--font-mono)
Tailwind v4	--font-sans: var(--font-inter);

⚠ COMMON MISTAKES

- Calling `Inter()` inside a component; loaders must run at module scope
- Loading the same font in several files; each call adds another copy
- Forgetting `weight` on a non-variable font; the build errors

✓ TIPS & BEST PRACTICES

- Export all fonts from one `fonts.ts` file and import them where needed
- Prefer variable fonts: one file covers every weight
- Set `preload: false` on fonts used only below the fold

QUICK SUMMARY

- `next/font/google` — self-hosted at build
- `next/font/local` — your own files
- `variable` — CSS custom property
- Module scope — call loaders once

Metadata API

export const metadata · generateMetadata() · title.template · openGraph · twitter · viewport

HEAD TAGS FROM LAYOUTS AND PAGES

Export a `metadata` object or an async `generateMetadata()` function from any `layout` or `page` (Server Components only) and Next.js renders the matching `<head>` tags. Metadata **merges** from the root down, so layouts set defaults and pages override them. `viewport` and `themeColor` live in a separate `viewport` export.

RULES

- Server Components only (not `'use client'` files)
- Shallow merge: nested objects like `openGraph` are replaced
- `fetch` calls in `generateMetadata` are memoized with the page
- Metadata can stream after the UI for bots that allow it

STATIC DEFAULTS + DYNAMIC PAGE

```tsx
// app/layout.tsx
import type { Metadata, Viewport } from 'next'
export const metadata: Metadata = {
  metadataBase: new URL('https://acme.com'),
  title: { default: 'Acme', template: '%s | Acme' },
  openGraph: { siteName: 'Acme', type: 'website' },
}
export const viewport: Viewport = { themeColor: '#5c6bc0' }
// app/blog/[slug]/page.tsx
export async function generateMetadata(
  props: PageProps<'/blog/[slug]'>): Promise<Metadata> {
  const { slug } = await props.params
  const post = await getPost(slug)                // deduped with page
  return { title: post.title,                     // 'Hello | Acme'
    openGraph: { images: [post.cover] },
    alternates: { canonical: `/blog/${slug}` } }
}
```

NOTES

- → `metadataBase` — makes relative URLs absolute
- → `template` — `%s` replaced by child titles
- → `title: { absolute: 'X' }` — ignore the template
- → Nested `openGraph` replaces the parent's entirely
- → Use `React.cache` for non-fetch data in both functions

COMMON FIELDS

FIELD	OUTPUT
`title`	`<title>`
`description`	`<meta name=description>`
`openGraph`	`og:*` tags
`twitter`	`twitter:card`, etc.
`alternates.canonical`	`<link rel=canonical>`
`robots`	`<meta name=robots>`
`icons`	`<link rel=icon>`

VIEWPORT EXPORT

FIELD	EXAMPLE
`width`	`'device-width'`
`initialScale`	`1`
`themeColor`	`[{ media, color }]`
`colorScheme`	`'light dark'`

⚠ COMMON MISTAKES

- Exporting `metadata` from a `'use client'` file; it's ignored with a build error
- Putting `themeColor` in `metadata`; it moved to the `viewport` export
- Forgetting `metadataBase`; relative Open Graph image URLs break in social previews

✓ TIPS & BEST PRACTICES

- Set `title.template` once in the root layout for consistent titles
- Share the data fetch between `generateMetadata` and the page via `React.cache`
- Repeat shared `openGraph` fields in pages that override it

QUICK SUMMARY

- `metadata` — static head tags
- `generateMetadata` — data-driven head tags
- `title.template` — consistent titles
- `viewport` — separate export

File-Based Metadata & SEO

SPECIAL METADATA FILES

Some metadata is easier as **files** placed in `app/`: icons, Open Graph images, `sitemap.xml`, `robots.txt`, and the web app manifest. Each can be a static file or a `.ts/.tsx` file that **generates** it. Generated Open Graph images use `ImageResponse` from `next/og`, which turns JSX into a PNG.

PLACEMENT

- Root `app/` files apply site-wide
- Segment files (e.g. `app/blog/[slug]/`) apply per route
- Generated files are static unless they use request data
- Metadata image `params` are a Promise in 16

DYNAMIC OG IMAGE

```tsx
// app/blog/[slug]/opengraph-image.tsx
import { ImageResponse } from 'next/og'
export const size = { width: 1200, height: 630 }
export const contentType = 'image/png'
export const alt = 'Blog post cover'
export default async function OG(
  { params }: { params: Promise<{ slug: string }> }) {
  const post = await getPost((await params).slug)
  return new ImageResponse(
    <div style={{ display: 'flex', width: '100%', height: '100%',
      alignItems: 'center', justifyContent: 'center',
      background: '#141720', color: 'white', fontSize: 64 }}>
      {post.title}
    </div>, size)
}
```

NOTES

→ `size`, `contentType`, `alt` — become meta tags

→ `ImageResponse` supports flexbox and a subset of CSS

→ Every `<div>` with several children needs `display: flex`

→ Load fonts as `ArrayBuffer` via the `fonts` option

SITEMAP.TS + ROBOTS.TS

```tsx
// app/sitemap.ts
import type { MetadataRoute } from 'next'
export default async function sitemap()
  : Promise<MetadataRoute.Sitemap> {
  const posts = await getPosts()
  return [{ url: 'https://acme.com', priority: 1 },
    ...posts.map((p) => ({
      url: `https://acme.com/blog/${p.slug}`,
      lastModified: p.updatedAt })),
  ]
}
// app/robots.ts
export default function robots(): MetadataRoute.Robots {
  return { rules: { userAgent: '*', allow: '/' },
    sitemap: 'https://acme.com/sitemap.xml' } }
```

METADATA FILES

FILE	GENERATES
`favicon.ico`	Root favicon
`icon.png / icon.tsx`	`<link rel=icon>`
`apple-icon.png`	Apple touch icon
`opengraph-image.tsx`	`og:image`
`twitter-image.tsx`	`twitter:image`
`sitemap.ts`	`/sitemap.xml`
`robots.ts`	`/robots.txt`
`manifest.ts`	`/manifest.webmanifest`

⚠ COMMON MISTAKES

- Using CSS grid or `display: block` layouts in `ImageResponse`; only flexbox is supported
- Hard-coding `localhost` URLs in the sitemap; use the production origin
- Placing `favicon.ico` in a nested segment; it only works at the `app/` root

✓ TIPS & BEST PRACTICES

- Split sitemaps over 50,000 URLs with `generateSitemaps()`
- Keep OG images simple and test them with a social preview tool
- Use `opengraph-image.alt.txt` for static OG image alt text

QUICK SUMMARY

- `opengraph-image.tsx` — generated OG images
- `ImageResponse` — JSX → PNG (`next/og`)
- `sitemap.ts` / `robots.ts` — SEO files
- `icon.*` / `manifest.ts` — app icons + PWA

Scripts & Lazy Loading

LOAD LESS, LATER

<Script> from next/script controls **when** third-party scripts load so they don't block rendering. next/dynamic (or React.lazy) splits a Client Component into its own chunk that loads only when rendered. @next/third-parties wraps common embeds (Google Analytics, Tag Manager, YouTube, Maps) with sensible defaults.

STRATEGIES

- `beforeInteractive` — before hydration (root layout only)
- `afterInteractive` — after hydration (default)
- `lazyOnload` — during browser idle time
- `worker` — web worker via Partytown (experimental)

SCRIPTS AND DYNAMIC IMPORTS

```
import Script from 'next/script'
import { GoogleAnalytics } from '@next/third-parties/google'

<Script src="https://js.stripe.com/v3" strategy="afterInteractive"
  onLoad={() => console.log('stripe ready')} />     // client component
<Script id="chat" strategy="lazyOnload">{`window.chat.init('abc')`}</Script>
<GoogleAnalytics gaId="G-XXXXXXX" />                // in root layout

'use client'
import dynamic from 'next/dynamic'
const Chart = dynamic(() => import('./chart'), {
  loading: () => <p>Loading chart…</p>,
})
const Map = dynamic(() => import('./map'), { ssr: false }) // browser only
export function Report({ show }: { show: boolean }) {
  return show ? <Chart /> : null                    // chunk loads on demand
}
```

NOTES

- → `onLoad` / `onReady` / `onError` need a Client Component
- → Inline scripts need a unique `id`
- → `ssr: false` is only allowed inside Client Components
- → Server Components are already code-split; `dynamic` is for client code
- → Import heavy libs inside event handlers: `await import('xlsx')`

@NEXT/THIRD-PARTIES

COMPONENT	FROM
GoogleAnalytics	@next/third-parties/google
GoogleTagManager	@next/third-parties/google
YouTubeEmbed	@next/third-parties/google
GoogleMapsEmbed	@next/third-parties/google

FINDING HEAVY CODE

TOOL	USE
next experimental-analyze	Turbopack analyzer (16.1+)
--output	Write report to .next/diagnostics
optimizePackageImports	Tree-shake barrel files
serverExternalPackages	Keep deps out of bundles

⚠ COMMON MISTAKES

- Using `ssr: false` in a Server Component; it's an error, so move the `dynamic()` call into a client file
- Loading analytics with `beforeInteractive`; it delays every page's interactivity
- Wrapping already-small components in `dynamic()`; each chunk adds a request

✓ TIPS & BEST PRACTICES

- Lazy-load modals, charts, and editors that aren't visible on first paint
- Run `next experimental-analyze` before and after a refactor to compare bundles
- Prefer `@next/third-parties` over hand-written GA snippets

QUICK SUMMARY

- `<Script strategy>` — control load timing
- `next/dynamic` — split client components
- `ssr: false` — browser-only (client files)
- `experimental-analyze` — inspect bundles

Authentication Patterns

WHO IS THE USER?

Authentication in the App Router centers on a **session**, stored either in an encrypted **stateless cookie** (a signed JWT) or as a **database session** referenced by a cookie ID. Wrap session reading in one `verifySession()` function inside a **Data Access Layer (DAL)** and call it wherever data is read or changed.

LIBRARY OPTIONS

- **Auth.js** (`next-auth` v5) — OAuth, email, credentials
- **Better Auth** — TypeScript-first, plugins, orgs
- **Clerk / WorkOS / Kinde** — hosted user management
- Roll your own with `jose` + `cookies()`

STATELESS SESSION + DAL

```ts
// lib/session.ts
import 'server-only'
import { SignJWT, jwtVerify } from 'jose'
import { cookies } from 'next/headers'
const key = new TextEncoder().encode(process.env.SESSION_SECRET)
export async function createSession(userId: string) {
  const token = await new SignJWT({ userId })
    .setProtectedHeader({ alg: 'HS256' }).setExpirationTime('7d').sign(key)
  ;(await cookies()).set('session', token, { httpOnly: true, secure: true,
    sameSite: 'lax', path: '/', maxAge: 60 * 60 * 24 * 7 })
}

// lib/dal.ts — imports cache (react), redirect (next/navigation)
export const verifySession = cache(async () => {
  const token = (await cookies()).get('session')?.value
  const res = token ? await jwtVerify(token, key).catch(() => null) : null
  if (!res) redirect('/login')
  const payload = res.payload
  return { userId: payload.userId as string }
})
```

NOTES

- → `server-only` — secret code can't leak to the client
- → `httpOnly` + `secure` + `sameSite` — baseline cookie safety
- → `cache()` — verify once per request, reuse everywhere
- → Call `createSession` from a Server Action after login
- → Logout: `(await cookies()).delete('session')`

SESSION TYPES

ASPECT	STATELESS (JWT)	DATABASE
Storage	Encrypted cookie	Sessions table
Lookup cost	None (verify signature)	DB query
Revoke instantly	✖ (wait for expiry)	✔
Size limit	~4 KB cookie	No limit
Good for	Simple apps	Security-sensitive apps

AUTH.JS V5 BASICS

PIECE	CODE
Config	`export const { auth, handlers } = NextAuth({...})`
Route	`app/api/auth/[...nextauth]/route.ts`
Read session	`const session = await auth()`
Sign in	`await signIn('github')`

⚠ COMMON MISTAKES

- Checking auth only in a layout; layouts don't re-run on every navigation, and data calls bypass them
- Storing roles or permissions in a long-lived JWT; changes don't apply until the token expires
- Hashing passwords with a fast hash (SHA-256); use `bcrypt` or `argon2`

✓ TIPS & BEST PRACTICES

- Return a minimal `{ userId, role }` from `verifySession()`; fetch the rest only when needed
- Rotate `SESSION_SECRET` with a key list so old sessions still verify during rollover
- Prefer a maintained auth library over custom crypto for OAuth flows

QUICK SUMMARY

- Session — cookie (JWT) or DB-backed
- `verifySession()` — one cached check
- DAL — auth checks live next to data access
- Auth.js / Better Auth — full-featured libraries

Protecting Routes & Data

DEFENSE IN LAYERS

Use **proxy** for fast, optimistic redirects based only on cookie presence. Do the **real check** close to the data: in the DAL, Server Components, Server Actions, and Route Handlers. Return **DTOs** (only the fields the UI needs) so sensitive columns never reach a Client Component.

WHERE TO CHECK

- proxy: cookie exists? (no DB calls)
- Page / component: `await verifySession()`
- Server Action: verify + authorize the record
- Route Handler: verify, return 401/403

DAL WITH AUTHORIZATION + DTO

```ts
// lib/dal/projects.ts
import 'server-only'
import { forbidden } from 'next/navigation'
import { verifySession } from '@/lib/dal'

export async function getProject(id: string) {
  const { userId, role } = await verifySession()
  const p = await db.project.findUnique({ where: { id },
    include: { members: true } })
  const canView = role === 'admin' || p?.members.some(m => m.userId === userId)
  if (!p || !canView) forbidden()                    // 403 UI
  return { id: p.id, name: p.name, updatedAt: p.updatedAt }  // DTO
}

// app/projects/[id]/page.tsx
export default async function Page(props: PageProps<'/projects/[id]'>) {
  const project = await getProject((await props.params).id)
  return <ProjectView project={project} />            // safe props
}
```

NOTES

- → Authorization uses the session, never a client-sent user ID
- → DTO drops fields like `billingEmail` or `apiKey`
- → `forbidden()` needs `experimental.authInterrupts`
- → `server-only` keeps the DAL out of client bundles

LAYER RESPONSIBILITIES

LAYER	CHECKS	CAN BE BYPASSED?
`proxy.ts`	Cookie present	Yes (matcher gaps)
`Layout`	Shell-level UI	Yes (no re-run)
`Page / component`	Session + render	Not for that page
`DAL function`	Session + ownership	No (single entry)
`Server Action`	Session + ownership	No

REACT TAINT APIS

API	PREVENTS
`experimental_taintObjectReference`	Passing a whole object to client
`experimental_taintUniqueValue`	Passing a specific secret value
`Enable`	`experimental.taint: true`

⚠ COMMON MISTAKES

- Doing a DB-backed auth check in proxy on every request; it slows every route and asset
- Passing a full user record to a Client Component; every field is visible in the page payload
- Relying on hidden UI for authorization; anyone can call the action or endpoint directly

✓ TIPS & BEST PRACTICES

- Funnel all reads through DAL functions so auth can't be forgotten
- Use `unauthorized()` (401) for logged-out users and `forbidden()` (403) for missing permissions
- Write a test per DAL function asserting that another user's ID is rejected

QUICK SUMMARY

- proxy — optimistic, cookie-only checks
- DAL — real auth + authorization
- DTOs — return only needed fields
- Taint APIs — block secrets from reaching clients

Security Headers & CSP

HARDENING RESPONSES

Add static security headers for every route through `headers()` in `next.config.ts`. A strict **Content Security Policy** with a per-request **nonce** has to be generated in `proxy.ts`; Next.js reads the nonce from the request's CSP header and applies it to its own scripts. Using nonces forces pages to render dynamically.

EXPOSURE RULES

- `NEXT_PUBLIC_*` env vars are inlined into client JS
- Everything else stays server-side
- Error messages from Server Components are hidden in production
- Source maps are off for the browser by default

STATIC HEADERS (NEXT.CONFIG.TS)

```ts
const security = [
  { key: 'Strict-Transport-Security',
    value: 'max-age=63072000; includeSubDomains' },
  { key: 'X-Content-Type-Options', value: 'nosniff' },
  { key: 'X-Frame-Options', value: 'DENY' },
  { key: 'Referrer-Policy',
    value: 'strict-origin-when-cross-origin' },
  { key: 'Permissions-Policy',
    value: 'camera=(), microphone=()' },
]
const nextConfig: NextConfig = {
  poweredByHeader: false,
  async headers() {
    return [{ source: '/(.*)', headers: security }]
  },
}
```

NONCE CSP (PROXY.TS)

```ts
export function proxy(req: NextRequest) {
  const nonce = btoa(crypto.randomUUID())
  const csp = [
    `default-src 'self'`,
    `script-src 'self' 'nonce-${nonce}' 'strict-dynamic'`,
    `style-src 'self' 'nonce-${nonce}'`,
    `img-src 'self' blob: data:`,
    `frame-ancestors 'none'`,
  ].join('; ')
  const headers = new Headers(req.headers)
  headers.set('x-nonce', nonce)
  headers.set('Content-Security-Policy', csp)
  const res = NextResponse.next({ request: { headers } })
  res.headers.set('Content-Security-Policy', csp)
  return res
}
// read it: (await headers()).get('x-nonce')
```

HEADER CHEAT SHEET

HEADER	PROTECTS AGAINST
`Content-Security-Policy`	XSS, injected scripts
`Strict-Transport-Security`	Protocol downgrade
`X-Frame-Options / frame-ancestors`	Clickjacking
`X-Content-Type-Options`	MIME sniffing
`Referrer-Policy`	URL leakage

ENV VAR EXPOSURE

VARIABLE	VISIBLE IN BROWSER?
`DATABASE_URL`	✖
`NEXT_PUBLIC_API_URL`	✔ (inlined at build)
`Server Action closure`	✖ (encrypted)
`Props to Client Components`	✔

⚠ COMMON MISTAKES

- Adding a nonce CSP and keeping pages static; nonces require dynamic rendering per request
- Prefixing secrets with `NEXT_PUBLIC_` to 'make them work' in the client; they're now public
- Forgetting to apply the CSP to the response as well as the forwarded request

✓ TIPS & BEST PRACTICES

- Exclude `_next/static`, `_next/image`, and prefetches from the CSP proxy matcher
- Start with `Content-Security-Policy-Report-Only` to find violations safely
- Set `poweredByHeader: false` to drop the `X-Powered-By` header

QUICK SUMMARY

- `headers()` — static security headers
- proxy nonce — strict per-request CSP
- `NEXT_PUBLIC_` — public by design
- `poweredByHeader: false` — hide the stack

Environment Variables

CONFIG THROUGH THE ENVIRONMENT

Next.js loads `.env*` files into `process.env` automatically. Variables are **server-only** by default. Prefixing a name with `NEXT_PUBLIC_` makes it available in the browser by **inlining its value at build time**, which means changing it later requires a rebuild. Runtime-config options were removed in 16; use env vars.

RULES

- `.env.local` is for secrets; keep it out of git
- `.env.local` is ignored when `NODE_ENV=test`
- Variables already set in the shell win over files
- Reference vars by full name: no dynamic lookups

DEFINING AND READING VARIABLES

```
# .env                       (committed defaults)
NEXT_PUBLIC_SITE_URL=https://acme.com
API_BASE=https://api.acme.com
# .env.local                 (secrets, not committed)
DATABASE_URL=postgres://user:pass@localhost:5432/acme
STRIPE_SECRET_KEY=sk_test_123
# Expansion
API_USERS=${API_BASE}/users

// server code
const db = new Pool({ connectionString: process.env.DATABASE_URL })
// client code — replaced at build with the literal string
const url = process.env.NEXT_PUBLIC_SITE_URL       // 'https://acme.com'
const bad = process.env['NEXT_PUBLIC_' + name]      // ✖ undefined
```

NOTES

- → `${VAR}` — expand another variable
- → Only literal `process.env.NEXT_PUBLIC_X` is inlined
- → Server code reads values at runtime (e.g. `next start`)
- → `@next/env` `loadEnvConfig()` loads them in scripts and tests
- → `experimental.typedEnv` generates types for env vars

LOAD ORDER (FIRST WINS)

#	SOURCE
1	`process.env` (shell / platform)
2	`.env.$(NODE_ENV).local`
3	`.env.local` (not in test)
4	`.env.$(NODE_ENV)`
5	`.env`

WHICH FILE?

FILE	COMMIT?	USE
`.env`	✔	Shared defaults
`.env.development`	✔	`next dev` defaults
`.env.production`	✔	`next build/start` defaults
`.env.local`	✖	Local secrets
`.env.test`	✔	Test runs

⚠ COMMON MISTAKES

- Changing a `NEXT_PUBLIC_` value on the host and expecting the client to update without a rebuild
- Committing `.env.local`; add `.env*.local` to `.gitignore`
- Reading `process.env.SECRET` in a Client Component; it's `undefined` there

✓ TIPS & BEST PRACTICES

- Validate env vars at startup with Zod (or `@t3-oss/env-nextjs`) so bad deploys fail fast
- For one image across environments, read config on the server at runtime and pass it down
- Commit a `.env.example` listing every required variable

QUICK SUMMARY

- `.env*` — loaded automatically
- `NEXT_PUBLIC_` — inlined into client at build
- `.env.local` — secrets, not committed
- Shell env — overrides every file

Databases & ORMs

TALKING TO A DATABASE

Server Components, Server Actions, and Route Handlers can query a database directly. Two things matter in Next.js: create **one client instance** per server process (hot reload in dev creates new modules, so cache it on `globalThis`), and use **pooling** or an HTTP-based **serverless driver** when deploying to functions that scale out.

WHERE QUERIES LIVE

- `lib/db.ts` — the client singleton
- `lib/dal/*.ts` — auth-aware query functions
- Mark both with `import 'server-only'`
- Never import them from `'use client'` files

PRISMA SINGLETON

```ts
// lib/db.ts
import 'server-only'
import { PrismaClient } from '@/generated/prisma/client'
import { PrismaPg } from '@prisma/adapter-pg'

const g = globalThis as unknown as { db?: PrismaClient }
const adapter = new PrismaPg({
  connectionString: process.env.DATABASE_URL })

export const db = g.db ?? new PrismaClient({ adapter })
if (process.env.NODE_ENV !== 'production') g.db = db

// usage in a Server Component
const posts = await db.post.findMany({
  where: { published: true }, take: 10 })
```

DRIZZLE + SERVERLESS DRIVER

```ts
// lib/db.ts
import 'server-only'
import { neon } from '@neondatabase/serverless'
import { drizzle } from 'drizzle-orm/neon-http'
import * as schema from './schema'

const sql = neon(process.env.DATABASE_URL!)
export const db = drizzle({ client: sql, schema })

// usage
import { eq } from 'drizzle-orm'
const user = await db.query.users.findFirst({
  where: eq(schema.users.id, id) })
```

CONNECTIONS BY HOST

DEPLOYMENT	APPROACH
Long-running server	Normal pool (e.g. `pg.Pool`)
Serverless functions	Pooler (PgBouncer, Supavisor) or HTTP driver
Edge (deprecated)	HTTP/WebSocket drivers only
Dev hot reload	`globalThis` singleton

CACHING QUERIES

GOAL	TOOL
Dedupe in one request	`React.cache(fn)`
Reuse across requests	`'use cache'` + `cacheTag`
Invalidate after write	`updateTag` / `revalidateTag`
Large result sets	Paginate; select needed columns

⚠ COMMON MISTAKES

- Creating `new PrismaClient()` in each file; dev hot reloads exhaust database connections
- Opening TCP connections per serverless invocation without a pooler; you hit connection limits
- Returning full ORM rows (with hashes and tokens) to Client Components

✓ TIPS & BEST PRACTICES

- Add `serverExternalPackages` for native drivers that shouldn't be bundled
- Run migrations in CI or a release step, never on app startup
- Use `select` / column lists so cached entries stay small

QUICK SUMMARY

- One client per process — `globalThis` in dev
- Pooling / HTTP drivers — for serverless
- `server-only` — keep DB code off the client
- `'use cache'` + tags — cache query results

Internationalization

ROUTING BY LOCALE

The App Router has no built-in i18n config (that was a Pages Router feature). Instead, put a `[lang]` dynamic segment at the top of `app/`, detect the preferred locale in **proxy**, and load per-locale **dictionaries** on the server. Libraries like **next-intl** package this pattern. Next.js 16.3 adds **root params** so any Server Component can read `lang` without prop drilling.

PIECES

- `app/[lang]/layout.tsx` — sets `<html lang>`
- `generateStaticParams` — prerender each locale
- proxy — redirect `/` to `/en` or `/de`
- Dictionaries — JSON loaded with `import()`

DICTIONARIES + LAYOUT

```tsx
// app/[lang]/dictionaries.ts
import 'server-only'
const dicts = {
  en: () => import('./dict/en.json').then(m => m.default),
  de: () => import('./dict/de.json').then(m => m.default),
}
export type Locale = keyof typeof dicts
export const getDictionary = (l: Locale) => dicts[l]()

// app/[lang]/layout.tsx
export const generateStaticParams = () =>
  [{ lang: 'en' }, { lang: 'de' }]
export default async function Layout(
  { children, params }: LayoutProps<'/[lang]'>) {
  const { lang } = await params
  return <html lang={lang}><body>{children}</body></html>
}
```

ROOT PARAMS + DETECTION

```tsx
// any Server Component (16.3)
import { lang } from 'next/root-params'
export async function Footer() {
  const dict = await getDictionary(
    (await lang()) as Locale)
  return <footer>{dict.footer.copyright}</footer>
}

// proxy.ts
const locales = ['en', 'de']
export function proxy(req: NextRequest) {
  const { pathname } = req.nextUrl
  if (locales.some((l) => pathname.startsWith(`/${l}`)))
    return
  const al = req.headers.get('accept-language') ?? ''
  const l = al.startsWith('de') ? 'de' : 'en'
  req.nextUrl.pathname = `/${l}${pathname}`
  return NextResponse.redirect(req.nextUrl)
}
```

LIBRARY OPTIONS

LIBRARY	HIGHLIGHTS
next-intl	Routing helpers, ICU messages, RSC support
Lingui	Extraction tooling, macros
i18next / react-i18next	Large ecosystem
DIY dictionaries	No dependency; server-only

FORMATTING BUILT IN

NEED	API
Numbers / currency	Intl.NumberFormat
Dates	Intl.DateTimeFormat
Relative time	Intl.RelativeTimeFormat
Plurals	Intl.PluralRules

⚠ COMMON MISTAKES

- Using the Pages Router `i18n` key in `next.config` for App Router routes; it doesn't apply
- Shipping every dictionary to the client; load on the server and pass only needed strings
- Forgetting to exclude `/api` and assets from the locale-redirect matcher

✓ TIPS & BEST PRACTICES

- Use `Intl` formatters with the active locale instead of formatting libraries
- Add `alternates.languages` in metadata for hreflang SEO
- Root params work inside `'use cache'`, and only the params read join the cache key

QUICK SUMMARY

- `app/[lang]/` — locale as a route segment
- proxy — detect and redirect
- `getDictionary()` — server-side strings
- `next/root-params` — read `lang` anywhere (16.3)

Unit & Component Testing

FAST TESTS FOR LOGIC AND UI

Use **Vitest** or **Jest** with **React Testing Library** for unit and component tests. Synchronous Server and Client Components can be rendered in jsdom. **Async Server Components** aren't supported by these tools yet, so test their data functions directly and cover full pages with end-to-end tests.

WHAT TO UNIT TEST

- DAL / data functions (mock the DB)
- Server Actions as plain async functions
- Client Components and hooks
- Pure utilities (formatters, validators)

VITEST SETUP

```
// vitest.config.mts
import { defineConfig } from 'vitest/config'
import react from '@vitejs/plugin-react'
import tsconfigPaths from 'vite-tsconfig-paths'

export default defineConfig({
  plugins: [tsconfigPaths(), react()],
  test: { environment: 'jsdom',
          setupFiles: ['./vitest.setup.ts'] },
})

// vitest.setup.ts
import '@testing-library/jest-dom/vitest'

// package.json → "test": "vitest"
```

COMPONENT TEST + ROUTER MOCK

```
import { render, screen } from '@testing-library/react'
import userEvent from '@testing-library/user-event'
import { vi, test, expect } from 'vitest'
import { SortSelect } from './sort-select'

const replace = vi.fn()
vi.mock('next/navigation', () => ({
  useRouter: () => ({ replace }),
  useSearchParams: () => new URLSearchParams('sort=new'),
  useParams: () => ({ category: 'shoes' }),
}))

test('updates the sort param', async () => {
  render(<SortSelect />)
  const select = screen.getByRole('combobox')
  await userEvent.selectOptions(select, 'price')
  expect(replace).toHaveBeenCalledWith('?sort=price')
})
```

JEST ALTERNATIVE

STEP	CODE
Config	`const createJestConfig = nextJest({ dir: './' })`
Import	`import nextJest from 'next/jest.js'`
Env	`testEnvironment: 'jsdom'`
Handles	SWC, CSS/images, `.env`, aliases

COMMON MOCKS

MODULE	MOCK
next/navigation	`useRouter`, `usePathname`, …
next/headers	`cookies()` returning a fake store
next/cache	`updateTag`, `revalidateTag` as `vi.fn()`
server-only	Empty module in test config

⚠ COMMON MISTAKES

- Rendering an async Server Component with RTL; it isn't supported, so test its data function or use E2E
- Forgetting to mock `next/navigation`; hooks throw outside the App Router context
- Importing a module with `server-only` in jsdom tests without aliasing it to an empty file

✓ TIPS & BEST PRACTICES

- Test Server Actions by calling them directly with a built `FormData`
- Use `screen.getByRole` queries; they match how users find elements
- Keep unit tests for logic and leave page wiring to Playwright

QUICK SUMMARY

- Vitest / Jest + RTL — component tests
- Async RSC — test data functions or E2E
- `vi.mock('next/navigation')` — router mocks
- `next/jest` — preconfigured Jest

End-to-End Testing

TESTING THE REAL APP

End-to-end tests drive a real browser against a running app, so they cover Server Components, Server Actions, streaming, and proxy together. Run them against a **production build** (`next build && next start`) to catch caching and prerender behavior. Next.js 16.3 adds `@next/playwright` with an `instant()` helper for navigation shells.

GOOD E2E TARGETS

- Auth flows and redirects
- Forms backed by Server Actions
- Loading and error states
- Critical purchase / signup journeys

PLAYWRIGHT.CONFIG.TS

```ts
import { defineConfig, devices } from '@playwright/test'

export default defineConfig({
  testDir: './e2e',
  use: { baseURL: 'http://localhost:3000',
         trace: 'on-first-retry' },
  webServer: {
    command: 'npm run build && npm run start',
    url: 'http://localhost:3000',
    reuseExistingServer: !process.env.CI,
    timeout: 180_000,
  },
  projects: [
    { name: 'chromium', use: devices['Desktop Chrome'] }],
})
```

A SERVER ACTION FLOW

```ts
import { test, expect } from '@playwright/test'

test('creates a todo', async ({ page }) => {
  await page.goto('/todos/new')
  await page.getByLabel('Title').fill('Write tests')
  await page.getByRole('button', { name: 'Add' }).click()
  await expect(page).toHaveURL(/\/todos\/\w+/)
  await expect(page.getByRole('heading'))
    .toHaveText('Write tests')
})

// 16.3: assert what shows instantly on navigation
import { instant } from '@next/playwright'
await instant(page, async () => {
  await page.click('a[href="/products/hats"]')
  await expect(page.locator('h1')).toContainText('Cap')
})
```

PLAYWRIGHT VS CYPRESS

ASPECT	PLAYWRIGHT	CYPRESS
Browsers	Chromium, Firefox, WebKit	Chromium, Firefox
Parallel runs	Built in	Paid / plugins
Multiple tabs	✔	✖
Component tests	Experimental	✔

NEXT.JS-SPECIFIC NOTES

TOPIC	ADVICE
Activity (16)	Hidden pages stay in DOM; use `getByRole`
Test API in prod	`experimental.exposeTestingApiInProductionBuild`
Seeding data	API route or DB script in `globalSetup`
Auth	Save `storageState` once, reuse

⚠ COMMON MISTAKES

- Testing only against `next dev`; caching, prefetching, and prerendering behave differently in production
- Using CSS selectors that match hidden routes kept alive by `<Activity>`; prefer role-based locators
- Logging in through the UI in every test; reuse a saved `storageState`

✓ TIPS & BEST PRACTICES

- Run E2E in CI against the same build you deploy
- Use `trace: 'on-first-retry'` and open traces with `npx playwright show-trace`
- Lock down instant navigations with `@next/playwright` `instant()` tests

QUICK SUMMARY

- Playwright — default E2E choice
- `webServer` — build + start before tests
- Role locators — robust with `<Activity>`
- `instant()` — test navigation shells (16.3)

Debugging

VS Code launch config · NODE_OPTIONS='--inspect' · server vs browser logs · React DevTools · DevTools MCP

FINDING THE PROBLEM FAST

Next.js code runs in two places, so logs and breakpoints do too: Server Components and Actions log to the **terminal**; Client Components log to the **browser console**. Attach a debugger to the server with `--inspect`, use the browser's DevTools for the client, and use the **Next.js DevTools** overlay for errors, routes, and cache insights.

BUILT-IN HELP

- Error overlay with source-mapped stacks
- Server Function calls logged in the dev terminal (16.2+)
- Next.js DevTools MCP for AI agents
- `next build --debug-prerender` for build errors

VS CODE LAUNCH.JSON

```
{
  "version": "0.2.0",
  "configurations": [
    { "name": "Next.js: server", "type": "node-terminal",
      "request": "launch", "command": "npm run dev -- --inspect" },
    { "name": "Next.js: client", "type": "chrome",
      "request": "launch", "url": "http://localhost:3000" },
    { "name": "Next.js: full stack", "type": "node", "request": "launch",
      "program": "${workspaceFolder}/node_modules/next/dist/bin/next",
      "runtimeArgs": ["--inspect"], "skipFiles": ["<node_internals>/**"],
      "serverReadyAction": { "action": "debugWithChrome",
        "pattern": "- Local:.+(https?://.+)", "uriFormat": "%s" } }
  ]
}
```

NOTES

- → `node-terminal` — breakpoints in Server Components and Actions
- → `chrome` — breakpoints in Client Components
- → Full stack — launches server, then opens Chrome attached
- → Or `next dev --inspect` / `next start --inspect` (16.2+)
- → `debugger;` statements work in both environments

WHERE DID IT LOG?

CODE	OUTPUT
`Server Component`	Terminal
`Server Action`	Terminal
`Client Component`	Browser console
`proxy.ts`	Terminal
`'use cache' function`	Terminal, prefixed Cache

DEBUGGING TOOLS

TOOL	USE
`React DevTools`	Props, state, Suspense panel
`Navigation Inspector`	Pause at the loading shell
`NEXT_PRIVATE_DEBUG_CACHE=1`	Verbose cache logs
`next info`	Environment for bug reports
`--experimental-cpu-prof`	CPU profiles in `.next-profiles/`

⚠ COMMON MISTAKES

- Looking for Server Component `console.log` output in the browser; it prints in the terminal
- Debugging a production-only bug in `next dev`; reproduce with `next build && next start`
- Ignoring `error.digest`; it's the key to the full server-side stack trace in logs

✓ TIPS & BEST PRACTICES

- Use `next build --debug-build-paths` to rebuild just the failing route
- Toggle Suspense and error boundaries in React DevTools to test fallback UI
- Connect your coding agent to the Next.js DevTools MCP for logs and route context

QUICK SUMMARY

- Server logs — terminal; client logs — browser
- `--inspect` — attach a Node debugger
- Next.js DevTools — overlay + insights
- `--debug-prerender` — readable build errors

Builds & Build Output

WHAT NEXT BUILD PRODUCES

`next build` compiles with **Turbopack**, type-checks, prerenders static routes, and writes everything to `.next/`. The terminal output shows timing for each step and a **route table** marking each route as static, partially prerendered, or dynamic. Since 16.3, Turbopack's **disk cache** also speeds up repeat builds by default.

16.X CHANGES

- Route table no longer lists JS bundle sizes
- `next build` no longer runs a linter
- Type checking can use TypeScript 7 (`tsc` CLI)
- Dev output goes to `.next/dev`, build to `.next/`

READING THE OUTPUT

```
$ next build
   ▲ Next.js 16.3.8 (Turbopack)
 ✓ Compiled successfully in 4.1s
 ✓ Finished TypeScript in 2.3s
 ✓ Collecting page data in 310ms
 ✓ Generating static pages (14/14) in 1.2s

Route (app)                          Revalidate  Expire
┌ ○ /                                        1h      1d
├ ○ /about
├ ● /blog/[slug]                            1d      1w
│   ├ /blog/hello
│   └ /blog/v16
├ ◐ /products/[id]
└ ƒ /dashboard

○ (Static)  ● (SSG)  ◐ (Partial Prerender)  ƒ (Dynamic)
```

READING IT

- → Timings show where build time goes
- → `Revalidate` / `Expire` come from `cacheLife`
- → ● lists the params from `generateStaticParams`
- → `ƒ` on a page you expected static? Look for `cookies()` or `searchParams`
- → Fix failures with `--debug-prerender`

INSIDE .NEXT/

PATH	CONTENTS
`.next/server`	Server bundles, prerendered HTML/RSC
`.next/static`	Hashed client assets (immutable)
`.next/cache`	Turbopack + image caches
`.next/standalone`	Minimal server (`output: 'standalone'`)
`.next/types`	Generated route types
`.next/dev`	`next dev` output (separate)

CI BUILD TIPS

TIP	HOW
Persist build cache	Cache `.next/cache` between runs
Build only some routes	`--debug-build-paths`
Skip type errors (risky)	`typescript.ignoreBuildErrors`
Consistent IDs	`generateBuildId` / `deploymentId`
Profile	`--experimental-cpu-prof`

⚠ COMMON MISTAKES

- Not caching `.next/cache` in CI; every build starts cold and takes longer
- Ignoring an unexpected `ƒ` in the route table; one dynamic API in a shared layout de-opts many routes
- Committing `.next/` to git; it's build output and should be ignored

✓ TIPS & BEST PRACTICES

- Compare the route table before and after caching changes
- Set the same `deploymentId` across instances to avoid version skew
- Use `next experimental-analyze` instead of the old bundle-size column

QUICK SUMMARY

- `next build` — Turbopack production build
- Route table — ○ ● ◐ ƒ
- `.next/cache` — persist in CI
- `--debug-prerender` — clearer build errors

Self-Hosting & Docker

RUNNING NEXT.JS YOURSELF

Any Node.js host can run `next build && next start` with **every feature** supported. For containers, set `output: 'standalone'`: the build copies only the files and dependencies needed into `.next/standalone`, producing a small image. When running **multiple instances**, configure a shared cache handler and a stable deployment ID.

SELF-HOSTING CHECKLIST

- Put a reverse proxy (nginx, Caddy) in front
- Serve `.next/static` with long cache headers or a CDN
- Image optimization uses `sharp` automatically
- Set `NEXT_SERVER_ACTIONS_ENCRYPTION_KEY` for many instances

STANDALONE DOCKERFILE

```
FROM node:22-alpine AS deps
WORKDIR /app
COPY package.json package-lock.json ./
RUN npm ci

FROM node:22-alpine AS build
WORKDIR /app
COPY --from=deps /app/node_modules ./node_modules
COPY . .
RUN npm run build                      # next.config: output: 'standalone'

FROM node:22-alpine AS run
WORKDIR /app
ENV NODE_ENV=production PORT=3000 HOSTNAME=0.0.0.0
COPY --from=build /app/.next/standalone ./
COPY --from=build /app/.next/static ./.next/static
COPY --from=build /app/public ./public
USER node
CMD ["node", "server.js"]
```

NOTES

- → `standalone` — traces only required `node_modules`
- → `static` and `public` must be copied manually
- → `server.js` replaces `next start` in standalone mode
- → `HOSTNAME=0.0.0.0` — listen outside the container
- → Pass runtime env vars at `docker run`, not in the image

MULTIPLE INSTANCES

CONCERN	SOLUTION
ISR / 'use cache' entries	`cacheHandlers` (e.g. Redis)
Server Action keys	`NEXT_SERVER_ACTIONS_ENCRYPTION_KEY`
Version skew	Same `deploymentId` per release
Static assets	Shared CDN or `assetPrefix`

RELEVANT CONFIG

OPTION	PURPOSE
`output: 'standalone'`	Minimal deployable folder
`cacheHandlers`	Custom `'use cache'` storage
`cacheMaxMemorySize`	In-memory cache size
`assetPrefix`	Serve assets from a CDN
`--keepAliveTimeout`	Match load balancer timeouts

⚠ COMMON MISTAKES

- Forgetting to copy `.next/static` into the standalone image; pages load without JS or CSS
- Baking secrets into the image with `ENV`; pass them at runtime instead
- Running several instances with the default in-memory cache and expecting shared invalidation

✓ TIPS & BEST PRACTICES

- Use `next start --keepAliveTimeout 70000` behind AWS ALB-style load balancers
- Run the container as a non-root user (`USER node`)
- Use the official `with-docker` example as a starting point

QUICK SUMMARY

- `next start` — full-feature Node server
- `output: 'standalone'` — small Docker images
- `cacheHandlers` — shared cache across instances
- Copy `.next/static` + `public` yourself

Static Export

HTML FILES, NO SERVER

With `output: 'export'`, `next build` writes plain HTML, CSS, and JS to `out/`. Host it on any static server or CDN (S3, GitHub Pages, Nginx). Server Components still run **at build time**, and Client Components work normally in the browser, but anything that needs a server at request time is unavailable.

WHAT STILL WORKS

- Server Components (rendered at build)
- Client-side fetching and navigation
- Route Handlers with `GET` (emitted as static files)
- `next/image` with a custom loader or `unoptimized`

CONFIGURE AND BUILD

```
// next.config.ts
const nextConfig: NextConfig = {
  output: 'export',
  trailingSlash: true,          // /about → /about/index.html
  images: { unoptimized: true },// no optimizer server
  // basePath: '/my-repo',      // GitHub Pages project site
}

// app/blog/[slug]/page.tsx — every param must be known at build
export async function generateStaticParams() {
  return (await getPosts()).map((p) => ({ slug: p.slug }))
}

$ next build                    # writes ./out
$ npx serve out                 # preview locally
```

NOTES

- → Dynamic routes need `generateStaticParams`
- → `trailingSlash` helps hosts that serve `folder/index.html`
- → `unoptimized` — or point `loaderFile` at an image CDN
- → `next start` doesn't serve an export; use a static server

NOT SUPPORTED

FEATURE	WHY
`cookies()` / `headers()`	No request at runtime
Server Actions	No server to POST to
`proxy.ts`	No request interception
Revalidation / ISR	Nothing to regenerate
`'use cache'` at runtime	Build-time only
Dynamic routes without params	Must be prerendered
Default image optimizer	Needs a server

HOSTING TARGETS

HOST	NOTE
GitHub Pages	Set `basePath` for project sites
S3 + CloudFront	Enable index document
Nginx	`try_files $uri $uri.html $uri/ =404`
Docker	`with-docker-export-output` example

⚠ COMMON MISTAKES

- Adding `cookies()` or a Server Action to an exported app; the build fails
- Forgetting `basePath` on GitHub Pages project sites; every asset path 404s
- Expecting new blog posts to appear without rebuilding; the export is frozen at build time

✓ TIPS & BEST PRACTICES

- Start static and remove `output: 'export'` later if you need server features
- Use client-side fetching (SWR) for data that changes between builds
- Trigger a rebuild from your CMS webhook to publish new content

QUICK SUMMARY

- `output: 'export'` — static `out/` folder
- `generateStaticParams` — required for dynamic routes
- `images.unoptimized` — or a custom loader
- No request-time features

Platforms & Deployment Adapters

CHOOSING WHERE TO DEPLOY

Next.js runs fully on any Node.js server or container. Platforms that run it differently use the **Deployment Adapter API** (`adapterPath`) to transform the build output. **Verified adapters** (currently Vercel and Bun) run the full Next.js test suite; Netlify and Cloudflare offer their own integrations while building verified adapters.

RUNTIME NOTES

- Node.js is the default runtime everywhere
- `runtime = 'edge'` segment config is deprecated
- `proxy.ts` always runs on Node.js
- `preferredRegion` is deprecated

ADAPTER CONFIG AND PLATFORM SETUP

```
// next.config.ts — custom adapter (platforms usually set this for you)
const nextConfig: NextConfig = {
  adapterPath: require.resolve('./my-adapter.js'),
}
// or zero-config on a platform:  NEXT_ADAPTER_PATH=/path/to/adapter

// Cloudflare Workers via OpenNext
$ npm i -D @opennextjs/cloudflare wrangler
$ npx opennextjs-cloudflare build && npx opennextjs-cloudflare deploy

// Vercel
$ npx vercel            # or connect the Git repo

// Netlify: built-in Next.js runtime, connect the repo
// Bun:     bun --bun next build && bun --bun next start
```

NOTES

- → Adapters hook build output: routes, prerenders, functions
- → Reference implementation: `nextjs/adapter-vercel`
- → OpenNext also targets AWS (SST)
- → Check each platform's support matrix for 16.x features

PLATFORM OVERVIEW

PLATFORM	APPROACH	STATUS
Vercel	Verified adapter	Full support
Bun	Verified adapter	Full support
Netlify	Own integration	Verified adapter in progress
Cloudflare	OpenNext	Verified adapter in progress
AWS Amplify / SST	Own integration	Varies
Node / Docker	next start	Full support

FEATURES NEEDING PLATFORM SUPPORT

FEATURE	NEEDS
'use cache' at runtime	Cache handler
Revalidation	Shared tag store
PPR shells	Serve shell + stream
Image optimization	Optimizer or loader
after()	waitUntil support

⚠ COMMON MISTAKES

- Choosing `runtime = 'edge'` for new routes; it's deprecated, and Node.js gets the full API surface
- Assuming every host supports Cache Components and PPR the same way; check the adapter's notes
- Hard-coding a platform's paths into app code; keep platform specifics in config

✓ TIPS & BEST PRACTICES

- Pick a verified adapter or plain `next start` when you need every feature
- Test your production build locally before switching platforms
- Read 'Deploying to Platforms' in the docs for the feature-by-feature matrix

QUICK SUMMARY

- `adapterPath` — Deployment Adapter API
- Verified — Vercel, Bun
- OpenNext — Cloudflare and AWS
- Node.js runtime — default; edge deprecated

Instrumentation & Observability

HOOKS FOR MONITORING

`instrumentation.ts` at the project root (or `src/`) exports `register()`, which runs **once** when each server instance starts: the place to initialize tracing and APM SDKs. `onRequestError()` is called for every server error with route and request context, which makes it easy to send errors to Sentry or your logger. `instrumentation-client.ts` runs in the browser before hydration.

FILES

- `instrumentation.ts` — server startup + errors
- `instrumentation-client.ts` — client analytics, error tracking
- Next.js emits OpenTelemetry spans for routing and fetch
- `NEXT_OTEL_VERBOSE=1` — more spans

OPENTELEMETRY + ERROR REPORTING

```ts
// instrumentation.ts
import { registerOTel } from '@vercel/otel'
import type { Instrumentation } from 'next'

export function register() {
  registerOTel({ serviceName: 'acme-web' })
}

export const onRequestError: Instrumentation.onRequestError = async (
  err, request, context) => {
  await fetch('https://logs.acme.com/errors', {
    method: 'POST',
    body: JSON.stringify({
      message: err instanceof Error ? err.message : String(err),
      path: request.path,                          // '/blog/hello'
      routeType: context.routeType,                // 'render' | 'action' | …
      routePath: context.routePath,                // '/app/blog/[slug]'
    }),
  })
}
```

NOTES

- → `register()` — runs once per server process
- → `@vercel/otel` works on any host, not only Vercel
- → `context.routerKind` — `'App Router'` or `'Pages Router'`
- → Import Node-only SDKs inside `if (process.env.NEXT_RUNTIME === 'nodejs')`
- → Errors from `error.tsx` digests match these reports

ONREQUESTERROR CONTEXT

FIELD	VALUES
`routerKind`	`'App Router'` · `'Pages Router'`
`routePath`	`'/app/blog/[slug]'`
`routeType`	`render` · `route` · `action` · `proxy`
`renderType`	`dynamic` · `dynamic-resume` (PPR)
`revalidateReason`	`on-demand` · `stale` · undefined

LOGGING CONFIG

OPTION	EFFECT
`logging.fetches.fullUrl`	Log full fetch URLs in dev
`logging.incomingRequests`	Toggle request logs
`NEXT_PRIVATE_DEBUG_CACHE`	Cache debug output
`instrumentation-client`	Client-side tracking

⚠ COMMON MISTAKES

- Initializing SDKs at the top of `instrumentation.ts` instead of inside `register()`
- Importing Node-only packages unconditionally; guard with `NEXT_RUNTIME` checks
- Throwing inside `onRequestError`; reporting must never crash the request path

✓ TIPS & BEST PRACTICES

- Use the Sentry or Datadog Next.js SDKs; they wire up both instrumentation files
- Attach `error.digest` to user-facing messages so support can find the log
- Export traces to any OTLP collector (Grafana, Honeycomb, Jaeger)

QUICK SUMMARY

- `register()` — startup hook
- `onRequestError` — central server error reporting
- `@vercel/otel` — OpenTelemetry setup
- `instrumentation-client.ts` — browser hooks

Pages Router Basics

THE ORIGINAL ROUTER

The **Pages Router** maps files in `pages/` directly to routes: `pages/about.tsx` is `/about`. Every page is a Client Component that is server-rendered and hydrated. It is still supported in 16.x and can run **alongside** `app/`, which is how most large codebases migrate. New features (Server Components, Cache Components) are App Router only.

KEY DIFFERENCES

- File = route (no `page.tsx` folder convention)
- `_app.tsx` wraps every page; `_document.tsx` shapes HTML
- Router hooks come from `next/router`
- Head tags via `next/head`, not Metadata API

PAGES, _APP, AND _DOCUMENT

```tsx
// pages/blog/[slug].tsx  →  /blog/hello
import Head from 'next/head'
import { useRouter } from 'next/router'
export default function Post() {
  const router = useRouter()
  const { slug } = router.query                    // string | string[]
  return <><Head><title>{`Post: ${slug}`}</title></Head><h1>{slug}</h1></>
}

// pages/_app.tsx — global layout, CSS, providers
import type { AppProps } from 'next/app'
import '@/styles/globals.css'
export default function App({ Component, pageProps }: AppProps) {
  return <Layout><Component {...pageProps} /></Layout>
}

// pages/_document.tsx — custom <html>/<body>, server only
import { Html, Head as DocHead, Main, NextScript } from 'next/document'
export default function Document() {
  return <Html lang="en"><DocHead /><body><Main /><NextScript /></body></Html>
}
```

NOTES

- → `router.query` — dynamic params and query string together
- → `query` is empty on the first render of static pages until hydration
- → Global CSS can only be imported in `_app.tsx`
- → `_document` has no data fetching or event handlers
- → Links: same `next/link` component in both routers

FILE → ROUTE

FILE	ROUTE
pages/index.tsx	/
pages/about.tsx	/about
pages/blog/[slug].tsx	/blog/:slug
pages/docs/[...all].tsx	/docs/*
pages/404.tsx	Custom 404
pages/api/hello.ts	/api/hello

NEXT/ROUTER API

MEMBER	PURPOSE
router.push(url)	Navigate
router.replace(url)	Navigate without history
router.query	Params + query
router.isReady	Query is populated
router.events	routeChangeStart, …

⚠ COMMON MISTAKES

- Importing `useRouter` from `next/navigation` in `pages/`; Pages Router uses `next/router`
- Reading `router.query` before `router.isReady` on static pages; values are empty at first
- Importing global CSS in a page component; only `_app.tsx` may import global CSS

✓ TIPS & BEST PRACTICES

- Keep using the Pages Router for stable legacy areas and add new routes in `app/`
- Move shared layout into `_app.tsx` with a per-page `getLayout` pattern
- Use `next/link` everywhere so links work across both routers

QUICK SUMMARY

- `pages/` — file = route
- `_app.tsx` — global wrapper
- `_document.tsx` — custom HTML shell
- `next/router` — Pages Router hooks

Pages Router Data Fetching

DATA THROUGH EXPORTED FUNCTIONS

Pages Router pages fetch data with special exported functions that run **only on the server**. `getStaticProps` runs at build time (plus background **ISR** with `revalidate`), `getServerSideProps` runs on every request, and `getStaticPaths` lists dynamic routes to prebuild. Results arrive as page **props**.

RETURN OPTIONS

- `{ props }` — pass data to the page
- `{ notFound: true }` — render 404
- `{ redirect: { destination, permanent } }`
- `revalidate: n` — ISR (getStaticProps only)

STATIC PATHS + ISR

```tsx
// pages/blog/[slug].tsx
import type { GetStaticPaths, GetStaticProps,
  InferGetStaticPropsType } from 'next'

export const getStaticPaths: GetStaticPaths = async () => ({
  paths: (await getTopSlugs()).map((slug) => ({ params: { slug } })),
  fallback: 'blocking',              // render unknown slugs on demand
})

export const getStaticProps = (async ({ params }) => {
  const post = await getPost(params!.slug as string)
  if (!post) return { notFound: true }
  return { props: { post }, revalidate: 60 }   // ISR every 60 s
}) satisfies GetStaticProps<{ post: Post }>

export default function Page(
  { post }: InferGetStaticPropsType<typeof getStaticProps>) {
  return <article>{post.title}</article>
}
```

NOTES

- → `fallback: 'blocking'` — SSR the first request, then cache
- → `revalidate: 60` — regenerate in the background
- → `satisfies` + `InferGetStaticPropsType` — typed props
- → On-demand ISR: `res.revalidate('/blog/x')` in an API route
- → Props must be JSON-serializable (no `Date`)

WHICH FUNCTION?

FUNCTION	RUNS	USE FOR
`getStaticProps`	Build + ISR	CMS, blogs, docs
`getServerSideProps`	Every request	Per-user, auth pages
`getStaticPaths`	Build	Dynamic static routes
`Client fetch (SWR)`	Browser	Private, live data

APP ROUTER EQUIVALENT

PAGES ROUTER	APP ROUTER
`getStaticProps`	async component + `'use cache'`
`getServerSideProps`	async component (dynamic)
`getStaticPaths`	`generateStaticParams`
`revalidate: 60`	`cacheLife({ revalidate: 60 })`
`fallback: false`	Previous model: `dynamicParams = false`

⚠ COMMON MISTAKES

- Returning `Date` objects in props; they aren't JSON-serializable, so convert to strings
- Using `getServerSideProps` for content that rarely changes; it skips the CDN on every request
- Exporting these functions from non-page files (components); they're ignored

✓ TIPS & BEST PRACTICES

- Use `fallback: 'blocking'` for large catalogs; prebuild only the top pages
- Set `Cache-Control` headers inside `getServerSideProps` for CDN caching
- Migrate one page at a time: each maps cleanly to an async Server Component

QUICK SUMMARY

- `getStaticProps` — build-time + ISR
- `getServerSideProps` — per-request SSR
- `getStaticPaths` — list dynamic pages
- `notFound` / `redirect` — return shapes

API Routes & Migrating to App Router

ENDPOINTS AND THE MIGRATION PATH

Pages Router **API routes** live in `pages/api` and use Node-style `req/res` objects. In the App Router they become **Route Handlers** using Web `Request/Response`. Both routers can run **in the same project**, so migrate incrementally: move one route at a time from `pages/` to `app/` and ship as you go.

COEXISTENCE RULES

- A URL can't exist in both `app/` and `pages/`
- `app/` routes take priority in the build check
- Navigating between routers is a full page load
- Shared components work in both (mind `'use client'`)

PAGES/API (BEFORE)

```ts
// pages/api/posts/[id].ts
import type { NextApiRequest, NextApiResponse }
  from 'next'

export default async function handler(
  req: NextApiRequest, res: NextApiResponse) {
  const id = req.query.id as string
  if (req.method === 'GET')
    return res.status(200).json(await getPost(id))
  if (req.method === 'DELETE') {
    await deletePost(id)
    return res.status(204).end()
  }
  res.setHeader('Allow', 'GET, DELETE')
  res.status(405).end()
}
```

APP/API ROUTE HANDLER (AFTER)

```ts
// app/api/posts/[id]/route.ts
type Ctx = RouteContext<'/api/posts/[id]'>

export async function GET(_: Request, ctx: Ctx) {
  const { id } = await ctx.params
  return Response.json(await getPost(id))
}

export async function DELETE(_: Request, ctx: Ctx) {
  const { id } = await ctx.params
  await deletePost(id)
  return new Response(null, { status: 204 })
}
// 405 for other methods is automatic
```

SUGGESTED MIGRATION ORDER

#	STEP
1	Upgrade to 16; run codemods
2	Add `app/layout.tsx` (from `_app` + `_document`)
3	Move simple static pages first
4	Convert data functions to async components
5	API routes → Route Handlers or Server Actions
6	`next/router` → `next/navigation` hooks
7	Remove `pages/` when empty

API MAPPING

PAGES ROUTER	APP ROUTER
`req.query`	`ctx.params` + `nextUrl.searchParams`
`req.body`	`await req.json()`
`res.status(n).json(d)`	`Response.json(d, { status: n })`
`res.revalidate(path)`	`revalidatePath(path)`
`next/head`	Metadata API
`getLayout pattern`	Nested `layout.tsx`

⚠ COMMON MISTAKES

- Leaving the same route in both `pages/` and `app/` ; the build fails with a conflict
- Porting every API route; internal ones used only by your UI are often simpler as Server Actions
- Forgetting that `useRouter` from `next/router` doesn't work in `app/`

✓ TIPS & BEST PRACTICES

- Run `npx @next/codemod@canary upgrade latest` before starting the migration
- Migrate the root layout first so new `app/` pages share the same shell
- Keep Pages Router routes for stable areas that don't need App Router features

QUICK SUMMARY

- `pages/api` — Node-style `req` / `res`
- Route Handlers — Web `Request` / `Response`
- Both routers coexist; migrate route by route
- Internal endpoints → Server Actions

Quick Reference — Routing & Rendering

CLI · special files · route segments · navigation · Server vs Client Components · data fetching · params · rendering modes

CLI

COMMAND	DOES	#
`npx create-next-app@latest`	new project	02
`next dev`	dev server (Turbopack)	05
`next build`	production build	51
`next start`	serve the build	51
`next lint / typegen`	tooling	05
`next.config.ts`	config file	04

SPECIAL FILES

FILE	DOES	#
`page.tsx`	route UI	03
`layout.tsx`	shared, persistent wrapper	08
`template.tsx`	wrapper that remounts	08
`loading.tsx`	Suspense fallback	10
`error.tsx`	error boundary (client)	10
`not-found.tsx`	404 UI	10
`route.ts`	API endpoint	30

ROUTE SEGMENTS

FOLDER	DOES	#
`app/blog/page.tsx`	/blog	06
`[slug]`	dynamic segment	07
`[...slug]`	catch-all	07
`[[...slug]]`	optional catch-all	07
`(group)`	group, no URL part	11
`_private`	not routable	11
`@modal / (.)photo`	parallel / intercepting	12

NAVIGATION

API	DOES	#
`<Link href="/a">`	client navigation	09
`prefetch={false}`	disable prefetch	09
`useRouter().push("/a")`	navigate (client)	09
`usePathname()`	current path	09
`useSearchParams()`	query string (client)	20
`redirect("/login")`	server redirect	09
`notFound()`	render not-found	10

SERVER VS CLIENT

ITEM	RULE	#
`Default`	Server Component	13
`"use client"`	top of file → Client	14
`Hooks, events, browser APIs`	Client only	14
`Data, secrets, DB`	Server	13
`Server → Client props`	must be serializable	15
`children prop`	nest Server in Client	15
`server-only package`	guard server code	13

DATA FETCHING

PATTERN	DOES	#
`async function Page()`	await in components	17
`await fetch(url)`	not cached by default	17
`Promise.all([…])`	parallel requests	17
`<Suspense fallback>`	stream a slow part	18
`use(promise)`	read promise in client	16
`SWR / React Query`	client fetching	19

PARAMS & SEARCHPARAMS

ITEM	DOES	#
`{ params }`	Promise — `await params`	20
`{ searchParams }`	Promise of query	20
`generateStaticParams()`	prebuild dynamic paths	21
`dynamicParams = false`	404 for others	21
`PageProps<"/blog/[slug]">`	typed props helper	20

RENDERING MODES

ITEM	DOES	#
`Static`	built at build time	21
`Dynamic`	per request (cookies, headers)	21
`cookies() / headers()`	request APIs (async)	33
`connection()`	opt into dynamic	33
`Partial Prerendering`	static shell + streamed holes	25
`output: "export"`	static HTML export	53

⚠ COMMON MISTAKES

- Adding `"use client"` to a whole tree — push it down to the leaf that needs interactivity
- Treating `params` / `searchParams` as plain objects — they are Promises; `await` them
- Importing server-only code (DB, secrets) into a Client Component

✓ TIPS & BEST PRACTICES

- Fetch in Server Components and pass data down as props
- Wrap slow sections in `<Suspense>` so the page streams
- Use route groups to give sections different layouts without changing URLs

QUICK SUMMARY

- Folders are routes; `page` / `layout` / `loading` / `error` files
- Server Components by default, `"use client"` to opt in
- `await params`, `await cookies()` — request APIs are async
- `<Link>` for navigation, `redirect()` on the server

Sheet 60

Quick Reference — Caching, Actions & Deployment

use cache · cacheLife & cacheTag · revalidation · Server Actions · forms · route handlers · proxy.ts · images & fonts · metadata · env · deploy

CACHING (CACHE COMPONENTS)

API	DOES	#
`cacheComponents: true`	enable in next.config	22
`"use cache"`	cache a function / component	22
`cacheLife("hours")`	set lifetime profile	23
`cacheTag("posts")`	tag for invalidation	23
`revalidateTag("posts")`	invalidate by tag	24
`revalidatePath("/blog")`	invalidate a route	24
`updateTag("posts")`	read-your-writes in actions	24

SERVER ACTIONS

ITEM	DOES	#
`"use server"`	mark action file / fn	27
`<form action={create}>`	call with FormData	28
`useActionState(fn, init)`	result + pending	28
`useFormStatus()`	pending in children	28
`useOptimistic()`	optimistic UI	28
`Validate + authorize`	actions are public endpoints	29

ROUTE HANDLERS

ITEM	DOES	#
`export async function GET(req)`	handler per method	30
`Response.json(data)`	JSON reply	30
`NextRequest / NextResponse`	helpers	30
`req.nextUrl.searchParams`	query	31
`await req.json()`	body	31
`{ params } (2nd arg)`	dynamic segment	31

PROXY.TS

ITEM	DOES	#
`export function proxy(req)`	runs before routes	32
`NextResponse.redirect()`	redirect	32
`NextResponse.rewrite()`	rewrite URL	32
`config.matcher`	which paths run it	32
`formerly middleware.ts`	renamed in Next 16	32

ASSETS & SEO

API	DOES	#
`<Image src w h alt>`	optimized images	37
`images.remotePatterns`	allow remote hosts	37
`next/font/google`	self-hosted fonts	38
`export const metadata`	static metadata	39
`generateMetadata()`	dynamic metadata	39
`sitemap.ts / robots.ts`	file-based SEO	40
`<Script strategy>`	third-party scripts	41

ENV & SECURITY

ITEM	DOES	#
`.env.local`	local secrets	45
`NEXT_PUBLIC_ prefix`	exposed to browser	45
`process.env.X`	server-side read	45
`headers() in config`	security headers	44
`CSP with nonce`	via proxy.ts	44
`Check auth in data layer`	not only in proxy	43

BUILD & DEPLOY

ITEM	DOES	#
`output: "standalone"`	minimal server bundle	52
`Dockerfile + standalone`	self-host	52
`output: "export"`	static site	53
`Adapters / Vercel`	managed platforms	54
`instrumentation.ts`	OpenTelemetry setup	55
`.next/`	build output	51

TESTING & PAGES ROUTER

ITEM	DOES	#
`Vitest / Jest + RTL`	unit & component	48
`Playwright`	end-to-end	49
`pages/ directory`	legacy router	56
`getServerSideProps`	per-request data	57
`getStaticProps`	build-time data	57
`pages/api/*.ts`	API routes	58

⚠ COMMON MISTAKES

- Assuming `fetch` is cached — in current Next it isn't unless you use `"use cache"`
- Server Actions without auth checks — anyone can POST to them
- Putting secrets in `NEXT_PUBLIC_` variables

✓ TIPS & BEST PRACTICES

- Tag cached data and call `revalidateTag` after writes
- Use `useActionState` for form errors and pending state
- Deploy with `output: "standalone"` for small Docker images

QUICK SUMMARY

- `"use cache"` + `cacheLife` + `cacheTag` control caching
- Server Actions handle forms; route handlers serve APIs
- `proxy.ts` runs before every matched request
- `next/image`, `next/font`, `metadata` for performance and SEO